

PRÉPARATION À LA SOUTENANCE

Analyse des Besoins & Préparation à la Soutenance

CRM PROBAR / CBI Tunisia — Support de révision jury (métier, architecture, base de données, sécurité, IA)

Fondé exclusivement sur l'analyse directe du code source et sur les quatre audits techniques réalisés sur ce même dépôt (Architecture, LLM/RAG/Providers, Sécurité/RBAC, Documentation complète)

Légende : **EXISTE** confirmé dans le code — **PARTIEL** — **NON IDENTIFIÉ** absent des sources analysées

Date de génération : 29 août 2026

Table des matières

1. Contexte du projet	
2. Identification des besoins	
3. Besoins spécifiques	
4. Besoins non spécifiques / génériques	
5. Comparaison spécifique vs non spécifique	
6. Exigences fonctionnelles	
7. Exigences non fonctionnelles	
8. Contraintes	
9. Règles métier	
10. Acteurs et utilisateurs	
11. Matrice rôles / besoins	
12. Matrice module / besoin	
13. Workflows métier	
14. Questions du jury — niveau métier	
15. Questions du jury — architecture	
16. Questions du jury — base de données	
17. Questions du jury — sécurité	
18. Questions du jury — intelligence artificielle	
19. Questions pièges du jury	
20. Choix technologiques	
21. « Pourquoi ? » — questions de soutenance	
22. « Comment ? » — questions de soutenance	
23. « Que se passe-t-il si ? »	
24. Questions à partir des diagrammes	
25. Simulation de soutenance	
26. Présentation orale (30 secondes par sujet)	
27. Réponses à ne pas donner	
28. Fiche de révision finale	
29. Réponses personnalisées au projet	
30. Conclusion	

1. Contexte du projet

Le code analysé (backend Node.js/Express/Sequelize « Backend Master », frontend Flutter « Dash Master Toolkit ») montre un CRM d'entreprise pour **CBI Tunisia / Probar Distribution**. Il ne s'agit pas d'un CRM commercial générique : le nombre et la nature des modules réellement présents (production PROMESH normalisée à 7 tables, table générique PROBAR/Mélange/Maintenance, suivi des récupérables déchet/produit fini) montrent une entreprise qui combine **distribution commerciale** (projets, contacts, pipeline de vente) et **suivi industriel d'atelier** (production, qualité, maintenance de machines).

Élément	Description
Problématique	Un ensemble de processus (projets, prospection commerciale, RH, maintenance, production industrielle) dispersés dans des écrans/formulaires séparés, chacun avec sa propre logique de permission — source : structure réelle du repository (17+ modules distincts confirmés en code)
Objectif	Centraliser ces processus dans une seule application (web + mobile + desktop), avec RBAC unifié et, pour un sous-ensemble de tâches, une interface conversationnelle (Copilot IA) — source : code + architecture observée
Utilisateurs	7 rôles réels confirmés dans l'ENUM Sequelize : <code>user</code> , <code>commercial</code> , <code>accueil</code> , <code>admin</code> , <code>superadmin</code> , <code>superadmin2</code> , <code>responsable_logistique_achat</code> — source : <code>src/models/User.js:15-20</code>
Départements	Commercial (contacts, pipeline), RH (congés/sorties), Maintenance (interne + industrielle), Logistique/Achat (production, POR PROMESH), Administration (utilisateurs, KPI globaux) — source : rôles + modules réels
Modules	Projets/Pipeline, Commercial Contacts, HR Requests, Maintenance Requests, Industrial Records (PROBAR/Mélange/Maintenance), POR PROMESH, Récupérables, Archive Requests, Kanban, Dashboard, Notifications, Google Calendar, Copilot IA — source : arborescence <code>src/modules/</code>
Processus informatisés	Tous les processus listés ci-dessus, avec workflows d'approbation réels (RH, Maintenance, Archive) et cycle brouillon→validation verrouillée (POR PROMESH) — source : code des services + ENUM de statut
Processus probablement manuels avant informatisation	Non identifié dans les sources analysées — aucune trace documentaire d'un état « avant » n'a été trouvée dans le dépôt ; toute affirmation sur une situation antérieure serait une supposition
Valeur métier	RBAC vérifié à 3 niveaux (agent, permission, service), traçabilité (ProjectActivity, HrRequestActivity, MfaAttemptLog), et un point d'entrée unique en langage naturel (Copilot) pour réduire la navigation manuelle entre modules — source : code + workflows PendingAction

Aucune affirmation sur « pourquoi » l'entreprise a commandé ce projet (contexte business, concurrence, ROI attendu) n'est faite ici au-delà de ce que le code lui-même permet de déduire — le jury doit s'attendre à ce que ces questions relèvent du contexte projet (cahier des charges), pas du code, et la réponse honnête est « non identifié dans les sources analysées » si le jury demande un chiffre d'affaires ou un objectif business précis non présent dans le dépôt.

2. Identification des besoins

ID	Besoin	Acteur	Module	Description	Source
BF-01	Créer/consulter un projet	Tout utilisateur	Projects	Assistant 5 étapes, pipeline visuel	Code + API (project.routes.js)
BF-02	Changer l'étape d'un projet	Propriétaire/admin-tier	Pipeline/Kanban	Déplacement kanban avec invalidation de cache	Code (requireOwnerOrAdmin)
BF-03	Gérer un contact commercial	Commercial/admin-tier	Commercial Contacts	Création, statut, historique	Code + Base de données (CommercialContact)
BF-04	Programmer une relance	Commercial	Commercial Contacts / Copilot	Relance datée, affectation commercial	Code (schedule_commercial_contact_relance) + Workflow PendingAction
BF-05	Soumettre une demande RH	Tout utilisateur	HR Requests	Congé ou autorisation de sortie	Code + Base de données (HrRequest)
BF-06	Traiter une demande RH	Gestionnaire RH désigné	HR Requests	Accepter/refuser/traiter	Code (requireHrReviewer)
BF-07	Signaler une panne	Tout utilisateur	Maintenance Requests	Équipement, urgence, description	Code + Base de données
BF-08	Gérer une demande de maintenance	Gestionnaire maintenance désigné	Maintenance Requests	Accepter/assigner/démarrer/clôturer	Code (requireMaintenanceManager)
BF-09	Saisir une fiche de production (PROBAR/Mélange/Maintenance machine)	Admin-tier + responsable logistique/achat	Industrial Records	Machine, poste, quantité ou panne	Code + Base de données (IndustrialRecord)
BF-10	Saisir/valider une fiche POR PROMESH	Admin-tier + responsable logistique/achat	POR PROMESH	Cycle brouillon → validation verrouillée	Code (isLocked) + API (/:id/validate)
BF-11	Saisir une fiche Récupérables	Admin-tier + responsable logistique/achat	Récupérables	Déchet/produit fini par machine/ligne/poste/date	Code + Base de données (RecuperableFiche/Ligne)
BF-12	Demander l'archivage/désarchivage d'un projet	Tout utilisateur (demande), root admin (décision)	Archive Requests	Workflow d'approbation	Code + Base de données (ArchiveRequest)
BF-13	Consulter des KPI	Tous (personnel), admin-tier (global)	Dashboard	Indicateurs agrégés	Code (kpiRoleGuard)
BF-14	Recevoir des notifications	Tous	Notifications	Temps réel (Socket.IO) + persistées	Code + Base de données
BF-15	Synchroniser un calendrier personnel	Tous (compte connecté)	Google Calendar	Événements liés à des relances/réunions	Code (googleCalendar.service.js)
BF-16	Administrer les comptes utilisateurs	Admin-tier	Utilisateurs	Activation/désactivation, rôle (superadmin(2) seul)	Code (admin.js)
BF-17	Poser une question en langage naturel	Tous rôles sauf accueil	Copilot IA	Question documentaire (RAG) ou donnée métier (Tool)	Code (src/ai/)
BF-18	Demander une action via le Copilot avec confirmation	Commercial/admin-tier (selon agent)	Copilot IA	PendingAction : proposition → confirmation → exécution	Code (PendingActionStore)
BF-19	Importer des clients (CSV)	Admin-tier	Clients	Import en masse	Code (client.routes.js)
BF-20	Authentification / session	Tous	Auth	JWT access/refresh, MFA (partiel)	Code (auth.routes.js)

3. Besoins spécifiques

Besoins directement liés au métier de l'entreprise, non réductibles à une fonctionnalité CRM générique. Chaque piste suggérée par le brief a été vérifiée dans le code avant d'être retenue.

ID	Besoin spécifique	Pourquoi spécifique ?	Acteur	Module	Implémentation
BS-01	Cycle brouillon → validation verrouillée POR PROMESH	Règle métier propre à la ligne de production PROMESH — un verrou irréversible n'existe dans aucun CRM générique	Resp. logistique/achat, admin-tier	POR PROMESH	<code>isLocked</code> (PorPromesh), <code>POST /:id/validate</code>
BS-02	Table générique polymorphe PROBAR/Mélange/Maintenance	Modélisation ad hoc (colonne <code>module</code> discriminante) reflétant 3 postes de travail réels d'un même atelier	Resp. logistique/achat, admin-tier	Industrial Records	<code>IndustrialRecord</code> (table <code>industrial_records</code>)
BS-03	Suivi déchet/produit fini par machine/ligne/poste	Calcul de rendement industriel propre à un atelier de production physique	Resp. logistique/achat	Récupérables	<code>RecuperableLigne.dechetKg</code> / <code>dechetProduitFiniKg</code>
BS-04	Unicité d'une fiche Récupérables par (module,machine,ligne,poste,date)	Règle de production réelle : une seule fiche par poste de travail et par jour	Resp. logistique/achat	Récupérables	Contrainte unique composite en base
BS-05	Rôle « responsable_logistique_achat » à périmètre HTTP restreint	Rôle métier propre à l'organisation (pas admin, pas commercial) avec accès limité à une liste fixe de préfixes	Responsable logistique/achat	Transverse	<code>moduleAccessGuard.js</code>
BS-06	Comptes désignés par email en dur (gestionnaire RH, maintenance, root admin)	Modèle hybride rôle-ou-email propre à l'organisation, pas un RBAC générique par rôle seul	Gestionnaires désignés	RH, Maintenance, Archive	<code>requireHrReviewer</code> , <code>requireMaintenanceManager</code> , <code>requireRootAdmin</code>
BS-07	Confirmation explicite avant toute écriture proposée par le Copilot	Garde-fou métier propre à ce projet contre l'exécution non supervisée d'une IA	Commercial, admin-tier	Copilot IA	<code>PendingActionStore</code>
BS-08	<code>requireToolUse</code> anti-hallucination	Correctif né d'un incident réel observé (le LLM affirmait avoir consulté une donnée sans l'avoir fait)	Système (transverse au Copilot)	Copilot IA	<code>toolCallingChain.js</code>
BS-09	Synchronisation Task CRM + Google Calendar pour les relances/réunions Copilot	Intégration métier propre reliant une action IA à un calendrier réel, avec idempotence	Commercial	Copilot IA / Calendrier	<code>multiRecipientCalendarSync.service.js</code> , migration <code>copilot_meeting</code>
BS-10	Auto-crédation d'action de suivi selon l'étape du pipeline (<code>autoCreateAction</code>)	Automatisation métier propre au processus de vente de l'entreprise	Commercial	Pipeline	<code>PipelineStage.autoCreateAction</code>

Analyse détaillée — BS-01 (Cycle POR PROMESH)

1. Pourquoi ce besoin existe-t-il ? Une fiche de production doit devenir infalsifiable une fois validée, pour garantir la fiabilité des statistiques de ligne. **2. Problème résolu :** empêcher une modification a posteriori d'une donnée de production déjà validée. **3. Utilisateur :** resp. logistique/achat, admin-tier. **4. Pourquoi un CRM générique ne suffit pas :** aucun CRM standard ne modélise un verrou métier irréversible sur un enregistrement industriel. **5. Traduction technique :** champ booléen `isLocked` vérifié côté service avant toute écriture. **6. Tables :** `por_promesh` + 6 tables liées. **7. API :** `POST /por-promesh/:id/validate`. **8. Règle métier :** BS-01 elle-même. **9. Écrans :** `por_promesh_dashboard_screen.dart`, `por_promesh_list_screen.dart`. **10. Résultat métier :** une fiche validée devient une source de vérité figée pour les statistiques de ligne.

Analyse détaillée — BS-07 (Confirmation Copilot)

1-2. Empêcher qu'un LLM (potentiellement peu fiable, cf. incidents documentés) exécute une écriture réelle sans supervision humaine. **3.** Commercial (relances), tout utilisateur (réunions), admin-tier (automatisation). **4.** Un CRM générique ne pose pas ce garde-fou pour une IA conversationnelle — c'est une réponse à un risque propre à l'intégration IA de ce projet. **5.** `PendingActionStore.propose()` stocke l'action, `consume()` atomique empêche la double exécution, la confirmation transite en langage naturel (« confirme »/« annule »), jamais un objet renvoyé par le client. **6.** Aucune table dédiée — état en mémoire volatile. **7.** `POST /ai-copilot/chat` (même endpoint que tout message). **8.** RG-07 (section 9 du document précédent). **9.** `CopilotPendingActionCard` (Flutter). **10.** Aucune action IA n'est jamais exécutée sans confirmation textuelle explicite classifiée déterministiquement.

4. Besoins non spécifiques / génériques

ID	Besoin générique	Description	Module	Acteur	Implémentation
BG-01	Authentification	Login email/mot de passe	Auth	Tous	JWT (access 15min, refresh 7j), bcrypt
BG-02	Déconnexion	Invalidation de session	Auth	Tous	POST /auth/logout (invalide refreshTokenHash)
BG-03	Gestion des utilisateurs	CRUD comptes, activation	Utilisateurs	Admin-tier	admin.js , users.routes.js
BG-04	Gestion des rôles	7 rôles ENUM	Transverse	Superadmin(2)	User.role
BG-05	Permissions / RBAC	Contrôle d'accès par rôle/email	Transverse	Tous	17 middlewares
BG-06	JWT	Jeton signé	Auth	Tous	jsonwebtoken
BG-07	MFA	Second facteur OTP	Auth	Tous (si branché)	PARTIEL — mfa.service.js jamais déclenché par signin
BG-08	Notifications	Personnelles, lu/non lu	Notifications	Tous	Socket.IO + Notification
BG-09	Recherche / filtres	Recherche par nom/statut	Plusieurs modules	Tous	Paramètres de requête (ex. search sur list_projects)
BG-10	Pagination	Listes paginées	Plusieurs modules	Tous	Paramètres page / pageSize
BG-11	Export PDF	Fiches/rapports	POR PROMESH, RH, Récupérables	Selon module	pdfkit , pdf2json
BG-12	Export Excel	Listes/rapports	Récupérables et autres	Selon module	exceljs , xlsx
BG-13	Upload de fichiers	Pièces jointes, justificatifs	RH, Maintenance, POR PROMESH, Devis	Selon module	multer (7 middlewares dédiés)
BG-14	Dashboard / KPI	Indicateurs	Dashboard	Tous/admin-tier	Agrégats Sequelize
BG-15	Historique / journal d'activité	Append-only	Projects, HR, Maintenance	Tous	ProjectActivity , HrRequestActivity
BG-16	Responsive	Desktop/tablette/mobile	Transverse (Flutter)	Tous	responsive_framework , 5 breakpoints
BG-17	Gestion des erreurs	Codes HTTP standards	Transverse	Tous	PARTIEL — pas de middleware d'erreur centralisé
BG-18	Logs	Traces applicatives	Transverse	Système	utils/logger.js + console.log (adoption partielle)

Chaque besoin générique reste nécessaire : sans authentification/RBAC solides, les besoins spécifiques (section 3) seraient inutilisables en toute sécurité — le générique est le socle sur lequel repose le spécifique.

5. Comparaison spécifique vs non spécifique

Critère	Besoin spécifique	Besoin non spécifique
Origine	Processus métier propre à CBI Tunisia (production, POR PROMESH)	Attendu de toute application d'entreprise
Métier	Directement lié au cœur de l'activité industrielle/commerciale	Support transverse, indépendant du secteur
Réutilisabilité	Faible hors de ce contexte industriel précis	Élevée (transposable à tout CRM)
Exemple CRM (générique)	—	Login, notifications, export PDF
Exemple projet (spécifique)	Verrouillage POR PROMESH, table polymorphe IndustrialRecord	—
Impact architecture	Justifie des modèles/services dédiés (por-promesh, industrial-records)	Justifie des couches transverses (middleware, services partagés)
Impact database	Tables/contraintes propres (unique composite, ENUM module)	Tables génériques (users, notifications)
Impact API	Endpoints dédiés à permission étroite (resp. logistique/achat)	Endpoints transverses (auth, users)

Les 10 besoins les plus spécifiques du projet

1. Cycle brouillon → validation verrouillée POR PROMESH (BS-01)
2. Table générique polymorphe IndustrialRecord (BS-02)
3. Suivi déchet/produit fini par machine/ligne/poste (BS-03)
4. Unicité de fiche Récupérables par combinaison (BS-04)
5. Rôle « responsable_logistique_achat » à périmètre restreint (BS-05)
6. Comptes désignés par email en dur (BS-06)
7. Confirmation explicite avant écriture IA (BS-07)
8. `requireToolUse` anti-hallucination (BS-08)
9. Synchronisation Task+Google Calendar pour relances/réunions IA (BS-09)
10. Auto-crédation d'action selon l'étape du pipeline (BS-10)

Les 10 besoins les plus génériques du projet

1. Authentification JWT (BG-01)
2. Déconnexion (BG-02)
3. Gestion des utilisateurs (BG-03)
4. RBAC (BG-05)
5. Notifications (BG-08)
6. Pagination (BG-10)
7. Export PDF (BG-11)
8. Export Excel (BG-12)
9. Upload de fichiers (BG-13)
10. Responsive (BG-16)

6. Exigences fonctionnelles

RF-001 — Créer un projet

ACTEUR: Utilisateur authentifié · **PRÉCONDITION:** session JWT valide · **DESCRIPTION:** saisie via assistant 5 étapes · **ENTRÉE:** nom, produit, localisation · **TRAITEMENT:** `POST /projects`, `INSERT Project` · **SORTIE:** projet créé, entrée pipeline étape par défaut · **RÈGLE MÉTIER:** aucun contrôle de rôle sur la création (route ouverte) · **EXCEPTION:** 400 si champs requis manquants · **MODULE:** Projects

RF-002 — Déplacer un projet dans le pipeline

ACTEUR: Propriétaire ou admin-tier · **PRÉCONDITION:** projet existant · **DESCRIPTION:** changement d'étape kanban · **ENTRÉE:** `pipelineStageld` · **TRAITEMENT:** `PUT /projects/:id/move-stage`, `requireOwnerOrAdmin` · **SORTIE:** étape mise à jour, cache kanban invalidé · **RÈGLE MÉTIER:** RG-08 (ownership) · **EXCEPTION:** 403 si ni propriétaire ni admin-tier · **MODULE:** Pipeline/Kanban

RF-003 — Soumettre une demande RH

ACTEUR: Tout utilisateur · **PRÉCONDITION:** aucune · **DESCRIPTION:** congé ou sortie · **ENTRÉE:** type, dates, motif · **TRAITEMENT:** `POST /hr-requests`, validation Joi conditionnelle · **SORTIE:** `HrRequest statut=en_attente` · **RÈGLE MÉTIER:** champs obligatoires varient selon `type` · **EXCEPTION:** 400 si schéma Joi invalide · **MODULE:** HR Requests

RF-004 — Traiter une demande RH

ACTEUR: Gestionnaire RH désigné · **PRÉCONDITION:** demande `en_attente` · **DESCRIPTION:** accepter/refuser/traiter · **ENTRÉE:** décision, commentaire · **TRAITEMENT:** `PUT /hr-requests/:id/accept|reject|process` · **SORTIE:** statut mis à jour, `HrRequestActivity` loggée · **RÈGLE MÉTIER:** `requireHrReviewer` (admin-tier + email désigné) · **EXCEPTION:** 403 hors habilitation · **MODULE:** HR Requests

RF-005 — Signaler une panne

ACTEUR: Tout utilisateur · **DESCRIPTION:** équipement, urgence, description, photos · **TRAITEMENT:** `POST /maintenance-requests`, `upload multer` (5 photos max) · **SORTIE:** `MaintenanceRequest statut=en_attente` · **MODULE:** Maintenance Requests

RF-006 — Valider une fiche POR PROMESH

ACTEUR: Admin-tier / resp. logistique-achat · **PRÉCONDITION:** fiche non verrouillée · **TRAITEMENT:** `POST /por-promesh/:id/validate` · **SORTIE:** `isLocked=true`, écriture définitivement bloquée · **RÈGLE MÉTIER:** BS-01 · **EXCEPTION:** refus si déjà verrouillée · **MODULE:** POR PROMESH

RF-007 — Saisir une fiche Récupérables

ACTEUR: Resp. logistique-achat, admin-tier · **ENTRÉE:** module, machine, ligne, poste, date, lignes(diamètre, déchet, produit fini) · **TRAITEMENT:** `POST /recuperables`, transaction `INSERT` fiche+lignes · **RÈGLE MÉTIER:** BS-04 (contrainte unique) · **EXCEPTION:** 409/erreur si combinaison déjà existante · **MODULE:** Récupérables

RF-008 — Demander l'archivage d'un projet

ACTEUR: Tout utilisateur · **TRAITEMENT:** `POST /archive-requests`, `statut=pending` · **SORTIE:** `ArchiveRequest` créée, notifiée au root admin · **MODULE:** Archive Requests

RF-009 — Approuver/rejeter un archivage

ACTEUR: Root admin (email en dur) · **TRAITEMENT:** `PUT /:id/approve|reject`, `requireRootAdmin` · **SORTIE:** `Project.isArchived` mis à jour · **MODULE:** Archive Requests

RF-010 — Programmer une relance commerciale via Copilot

ACTEUR: Commercial, admin-tier · **PRÉCONDITION:** contact et date nommés dans le message · **TRAITEMENT:** extraction déterministe → résolution serveur (jamais le LLM) → `PendingActionStore.propose()` → confirmation → `schedule_commercial_contact_relance` · **SORTIE:** relance créée + synchro calendrier best-effort · **RÈGLE MÉTIER:** BS-07, ownership contact vérifiée en base · **MODULE:** Commercial Contacts / Copilot

RF-011 — Poser une question documentaire au Copilot

ACTEUR: Tous rôles sauf accueil · **TRAITEMENT:** `IntentDetector` → `CrmAssistantAgent` → RAG (recherche cosinus sur documentation) → réponse ancrée avec citations · **SORTIE:** réponse + citations · **MODULE:** Copilot IA

RF-012 — Consulter mes KPI

ACTEUR: Tous · **TRAITEMENT:** GET /dashboard/user (sans kpiRoleGuard, volontairement) · **MODULE:** Dashboard

RF-013 — Consulter les KPI globaux

ACTEUR: Admin-tier · **TRAITEMENT:** GET /dashboard/kpis|professional , kpiRoleGuard · **EXCEPTION:** 403 pour un rôle non admin-tier · **MODULE:** Dashboard

RF-014 — Réinitialiser un mot de passe

ACTEUR: Tout compte · **TRAITEMENT:** token SHA-256 haché, TTL 15 min, réponse neutre anti-énumération · **MODULE:** Auth

RF-015 — Changer le rôle d'un utilisateur

ACTEUR: Superadmin/superadmin2 uniquement · **TRAITEMENT:** PUT /admin/users/:id/role · **RÈGLE MÉTIER:** RG-06 · **EXCEPTION:** 403 pour admin simple · **MODULE:** Utilisateurs

7. Exigences non fonctionnelles

ID	Exigence	Critère	Justification	Implémentation
RNF-01	Performance LLM bornée	Timeout 240 000ms (Ollama)	Pire cas réel mesuré à 179 681ms (CPU seul, sans GPU)	<code>ollamaProvider.js</code>
RNF-02	Anti-boucle Tool Calling	5 tours maximum	Éviter une boucle infinie du LLM	<code>maxToolCalls</code>
RNF-03	Disponibilité email	Jamais de perte	File d'attente avec retry (5 tentatives, backoff)	<code>emailQueue.service.js</code>
RNF-04	Sécurité mots de passe	Hachage bcrypt	Standard de l'industrie	<code>auth.routes.js</code>
RNF-05	Confidentialité des secrets	Jamais loggés	Clé API jamais transmise dans une réponse d'erreur	<code>providers/errors.js</code>
RNF-06	Intégrité référentielle	Clés étrangères + CASCADE/SET NULL	Cohérence des suppressions en cascade	Migrations Sequelize
RNF-07	Traçabilité	Journal d'activité append-only	Audit des changements de statut	<code>ProjectActivity</code> , <code>HrRequestActivity</code> , <code>MfaAttemptLog</code>
RNF-08	Idempotence de synchronisation calendrier	Clé (entityType,entityId,userId)	Éviter les doublons d'événements Google Calendar	<code>upsertSync</code> (testé)
RNF-09	Responsive multi-device	5 breakpoints	Usage terrain (tablette en atelier, mobile)	<code>responsive_framework</code>
RNF-10	Maintenabilité IA	Ports & Adapters	Changer de provider LLM sans toucher aux agents	<code>ILlmProvider</code>
RNF-11	Non-régression	Suite de tests Jest	~92% de couverture sur le kernel IA (audit préexistant)	<code>test/</code>
RNF-12	Compatibilité multiplateforme	6 plateformes natives	Usage web + mobile + desktop	Dossiers android/ios/web/windows/macOS/linux
RNF-13	Anti-hallucination	Fondé sur l'état réel, jamais le texte du LLM	Incidents réels observés (PROMESH, projets)	<code>requireToolUse</code>
RNF-14	Scalabilité horizontale IA	NON ATTEINTE	État conversationnel en mémoire process, non partagé	<code>PendingActionStore</code> (limite connue)

8. Contraintes

Contrainte	Origine	Impact	Solution / état
Aucune GPU disponible pour le LLM	Infrastructure	Latence variable (facteur ~20×)	Timeout élevé (240s) + plafond de génération (num_predict=768)
LLM local non fiable à 100% sur le protocole JSON	Modèle (qwen3:8b)	Risque d'hallucination de tool-calling	<code>requireToolUse</code> (3 agents), non généralisé à tous
Base de données PostgreSQL obligatoire	Choix technique (Sequelize)	Dépendance à un serveur PostgreSQL	Migrations sequelize-cli
Pas de base vectorielle dédiée disponible en environnement local	Infrastructure / choix de phase	RAG en mémoire, non persistant	Interface <code>IVectorStore</code> prête pour un futur adaptateur (pgvector)
Stockage fichiers local uniquement	Infrastructure	Pas de scalabilité multi-serveur pour les uploads	Répertoire <code>uploads/</code> disque
7 rôles fixes définis par ENUM Sequelize	Base de données	Ajout d'un rôle nécessite une migration	Migrations dédiées déjà utilisées pour ajouter des rôles (ex. superadmin2)
Comptes désignés par email en dur	Choix métier	Changement de gestionnaire nécessite une modification de config	<code>src/config/hrManager.js</code> , etc.
Rate limiting non systématique	Sécurité (dette identifiée)	Endpoints coûteux non protégés contre l'abus	Limité à auth/geocode/import
Pas de PM2 versionné	DevOps	Configuration d'exploitation non reproductible depuis le dépôt	Référencé en exploitation, non commité
Aucun Docker	DevOps	Environnement de déploiement non conteneurisé	Node.js exécuté directement
Kernel IA jamais autorisé à contourner le RBAC CRM	Sécurité (contrainte de conception)	Chaque Tool doit repasser par les services existants	Vérifié mécaniquement par tests policy
Compatibilité 6 plateformes Flutter	Choix technique	Contraintes UI transverses (responsive)	<code>responsive_framework</code>

9. Règles métier

ID	Règle	Condition	Action	Résultat	Module
RB-001	Ownership relance commerciale	<code>contact.createdBy !== actingUser.id</code> et rôle non admin-tier	Comparaison en base avant écriture	Refus (forbidden)	Commercial Contacts
RB-002	Verrouillage POR PROMESH	Fiche déjà validée (<code>isLocked</code>)	Blocage de toute écriture	Erreur / refus	POR PROMESH
RB-003	Clôture automatique Récupérables	Fiche ouverte >6 jours ou clôture manuelle	Mise à jour statut	<code>statut=cloturee</code>	Récupérables
RB-004	Unicité fiche Récupérables	Même (module,machine,ligne,poste,date)	Contrainte unique DB	Erreur d'insertion si doublon	Récupérables
RB-005	Activation de compte	Nouveau compte créé	<code>isActive=false</code> par défaut	Connexion refusée jusqu'à activation	Utilisateurs
RB-006	Changement de rôle réservé	Modification de <code>role</code>	Vérification <code>requireRole</code>	Réservé superadmin/superadmin2	Utilisateurs
RB-007	Confirmation explicite Copilot	PendingAction proposée	Classification déterministe du message suivant	Exécution uniquement si « confirme »	Copilot IA
RB-008	Restriction de préfixe pour un rôle	<code>role==="responsable_logistique_achat"</code>	Vérification <code>moduleAccessGuard</code>	403 hors préfixes autorisés	Transverse
RB-009	Déduplication des rappels de relance	Relance déjà notifiée le jour même	Vérification avant envoi	Pas de notification dupliquée	Cron (followup.job.js)
RB-010	Anti-hallucination Tool Calling	<code>requireToolUse=true</code> et <code>steps.length===0</code>	Rejet du <code>final_answer</code>	Message honnête de repli	Copilot IA

10. Acteurs et utilisateurs

Acteur	Rôle	Responsabilité	Modules	Permissions
Utilisateur standard	user	Consulte ses projets, soumet RH/Maintenance	Projects, HR, Maintenance, Notifications, Copilot	Lecture propre + soumission
Commercial	commercial	Gère contacts, relances, pipeline	Commercial Contacts, Pipeline, Copilot	KPI personnel, relance (PendingAction)
Accueil	accueil	Accès de base	Base uniquement	Exclu de l'accès Copilot
Responsable logistique/achat	responsable_logistique_achat	Production, POR PROMESH, Récupérables, Maintenance	Industrial Records, POR PROMESH, Récupérables, Maintenance, HR (historique)	Périmètre HTTP restreint (moduleAccessGuard)
Admin / Superadmin / Superadmin2	admin-tier	Administration globale	Tous	KPI globaux, changement de rôle (superadmin(2) seul)
Root admin (email désigné)	Aucun rôle ENUM dédié — email en dur	Approbation archivage	Archive Requests	requireRootAdmin
Gestionnaire RH désigné	Admin-tier + email désigné	Traitement des demandes RH	HR Requests	requireHrReviewer
Gestionnaire maintenance désigné	Admin-tier + responsable_logistique_achat + email désigné	Traitement des demandes de maintenance	Maintenance Requests	requireMaintenanceManager
Copilot IA (agent logiciel)	N/A	Agit toujours au nom d'un actingUser réel	Transverse	Jamais de privilège propre

Utilisateurs Finance : **NON IDENTIFIÉ** — aucun rôle ni module Finance transactionnel. **Utilisateurs industriels :** confirmés (resp. logistique/achat).

Utilisateurs IA : tous rôles sauf accueil .

11. Matrice rôles / besoins

Besoin	Admin-tier	Resp. logistique/achat	Commercial	User	Accueil
Créer un projet	Oui	Oui	Oui	Oui	Oui
Changer l'étape d'un projet	Oui	Si propriétaire	Si propriétaire	Si propriétaire	Si propriétaire
Gérer un contact commercial	Oui	Non	Oui	Non	Non
Traiter une demande RH	Oui (si désigné)	Non	Non	Non	Non
Gérer une demande de maintenance	Oui (si désigné)	Oui (si désigné)	Non	Non	Non
Saisir fiche Industrial Records / POR PROMESH	Oui	Oui	Non	Non	Non
Saisir fiche Récupérables	Oui	Oui	Non	Non	Non
Approuver archivage	Root admin seul	Non	Non	Non	Non
KPI globaux	Oui	Non	Non	Non	Non
Changer un rôle	Superadmin(2) seul	Non	Non	Non	Non
Utiliser le Copilot IA	Oui	Oui	Oui	Oui	Non

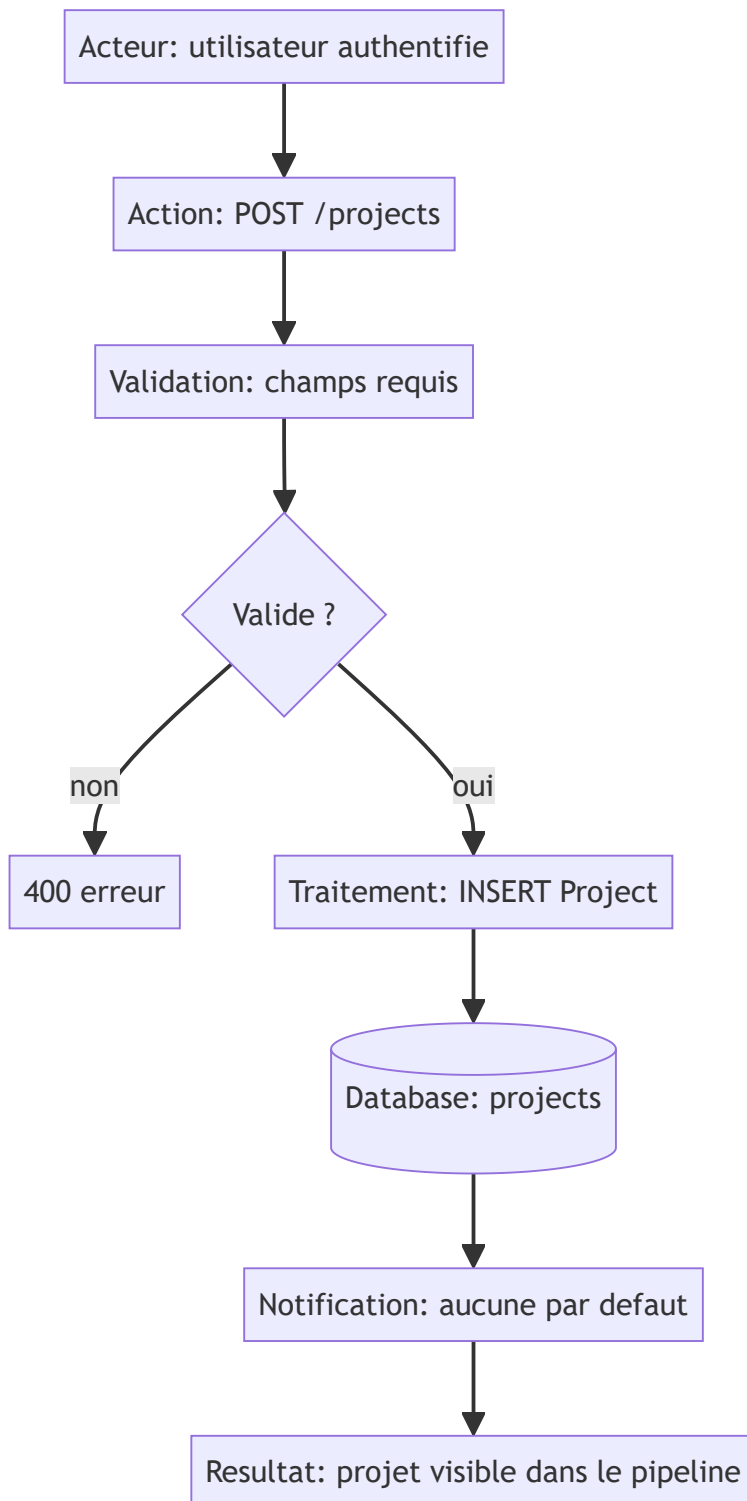
12. Matrice module / besoin

Module	Besoins spécifiques	Besoins génériques
Dashboard	—	KPI, pagination
Projects	Auto-cr�ation d'action selon �tape (BS-10)	CRUD, historique, upload devis
Pipeline	Auto-cr�ation d'action (BS-10)	—
Commercial	Relance via Copilot avec confirmation (BS-07)	CRUD contact, KPI
Maintenance (interne)	Comptes d�sign�s par email (BS-06)	Signalement, historique, upload photos
HR	Comptes d�sign�s par email (BS-06)	Soumission, historique, upload justificatifs
Production (Industrial Records)	Table polymorphe (BS-02)	CRUD
POR PROMESH	Cycle verrouillage (BS-01)	CRUD, export PDF
PROBAR	Couvert par BS-02 (module g�n�rique)	—
Production Records	Correspond � IndustrialRecord/PorPromesh	—
Production Summary	Non identifi� comme module distinct	—
Recoverables	Unicit� fiche (BS-04), suivi d�chet/produit fini (BS-03)	Historique, statistiques, export PDF/Excel
Finance	Non identifi� Non identifi�	
Documents	—	Upload local (sans authentification sur /uploads — dette identifi�e)
Import	—	Import CSV clients
Copilot IA	Confirmation explicite (BS-07), requireToolUse (BS-08), sync calendrier (BS-09)	R�ponse documentaire (RAG)

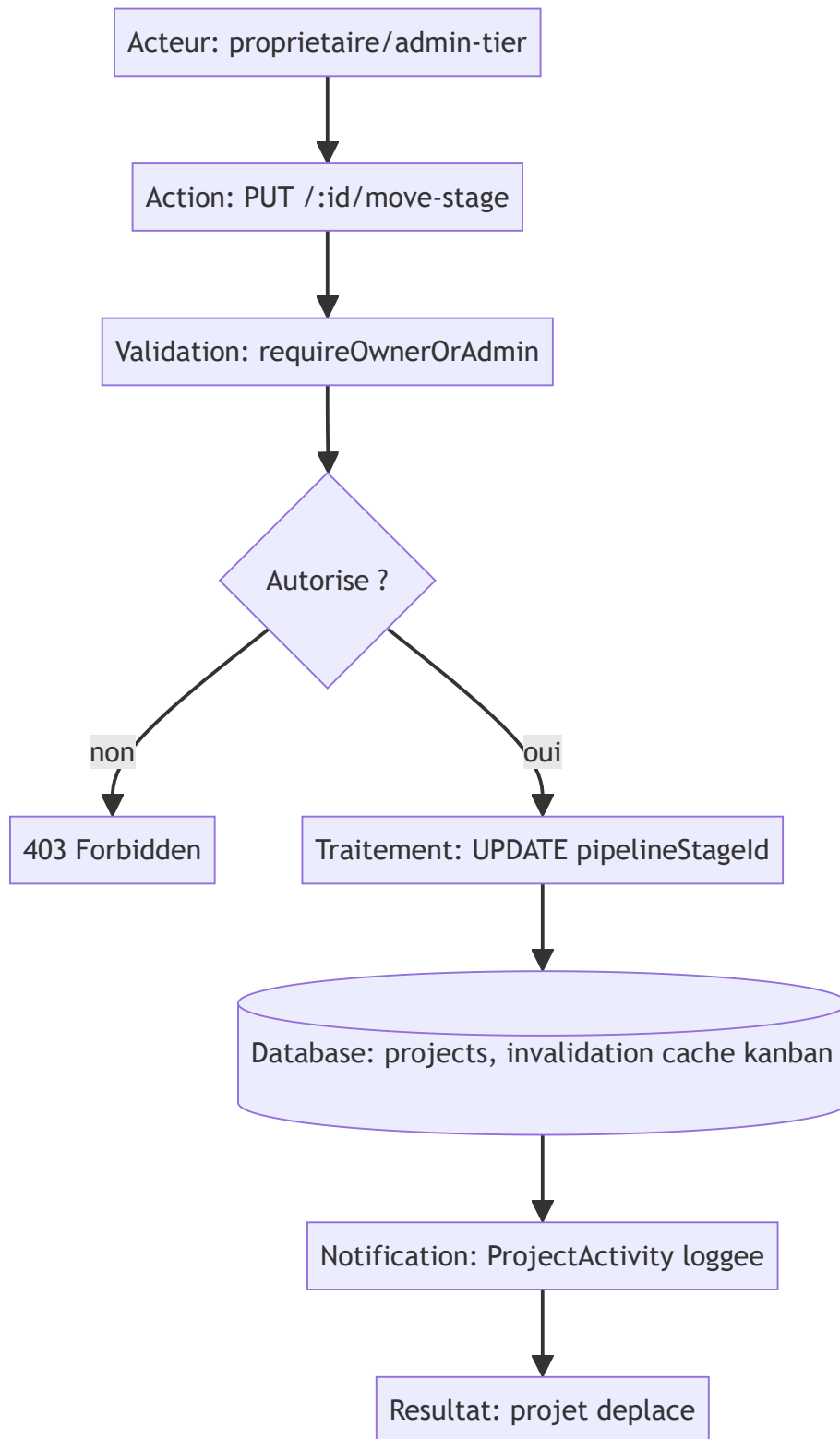
13. Workflows métier

Diagrammes d'activité détaillés (format Acteur→Action→Validation→Décision→Traitement→Database→Notification→Résultat). Cas très proches groupés pour éviter la répétition quasi identique.

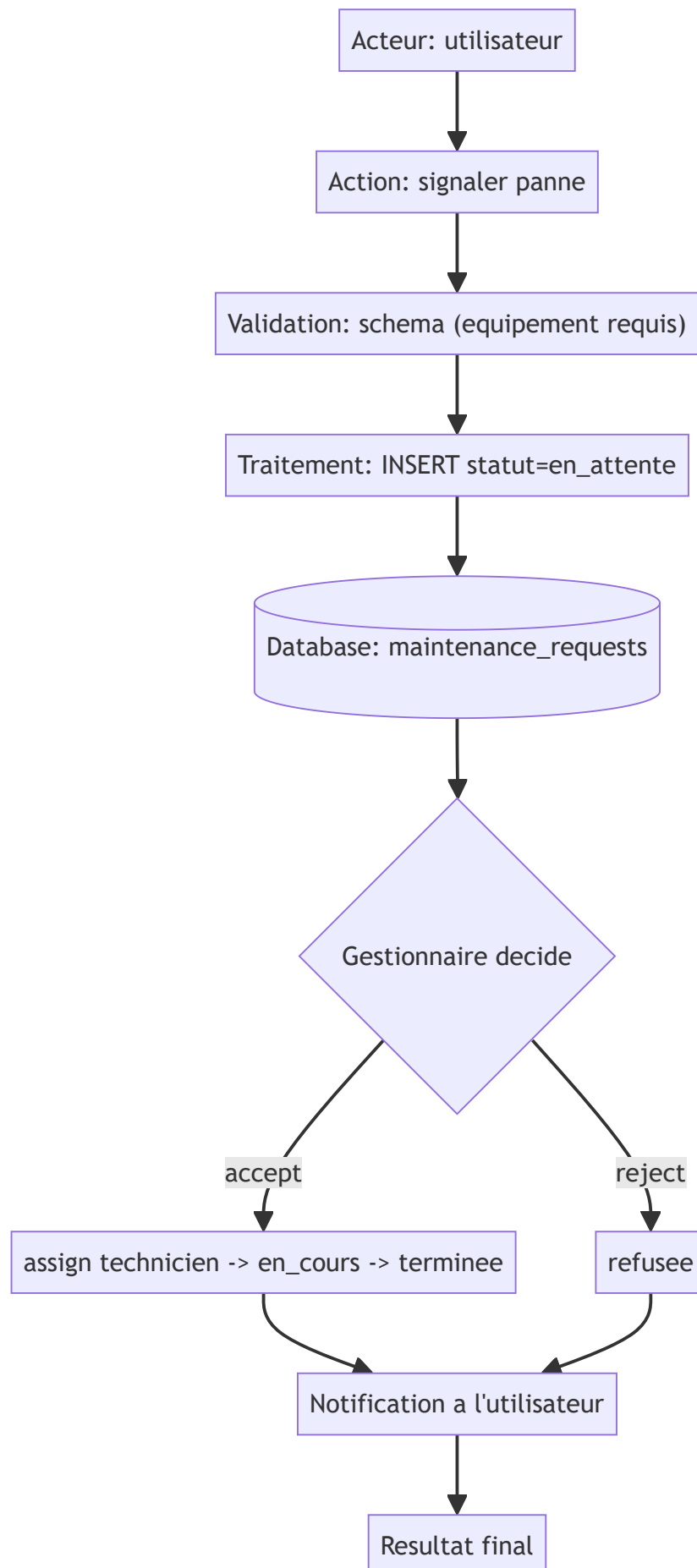
13.1 Création de projet



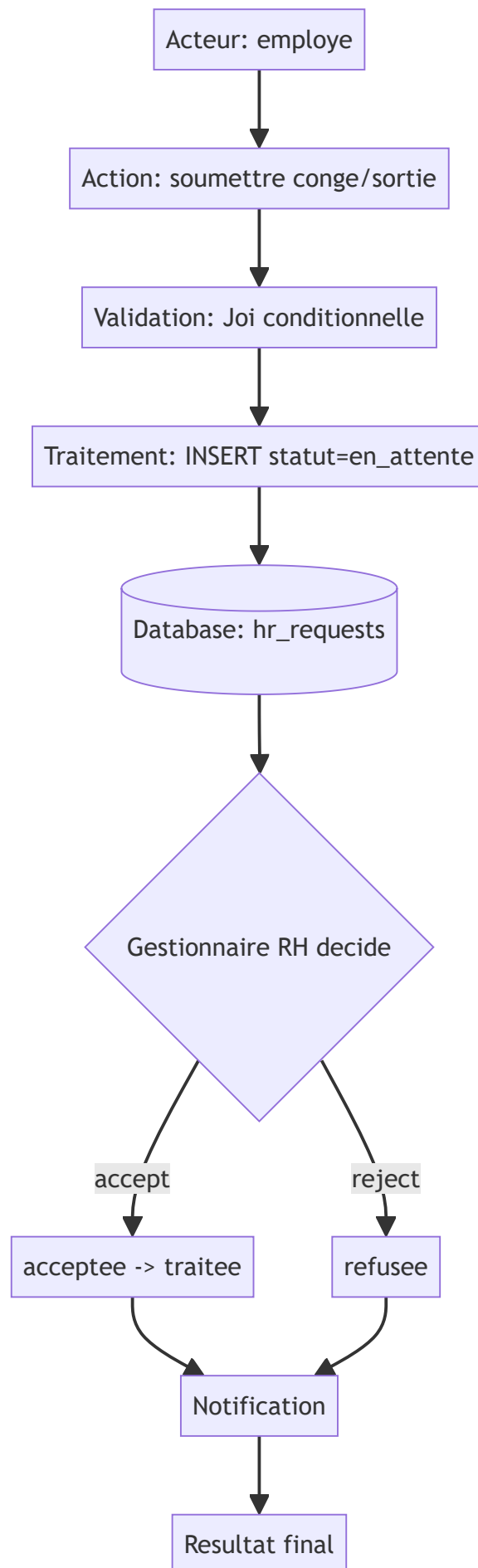
13.2 Suivi de projet (changement d'étape)



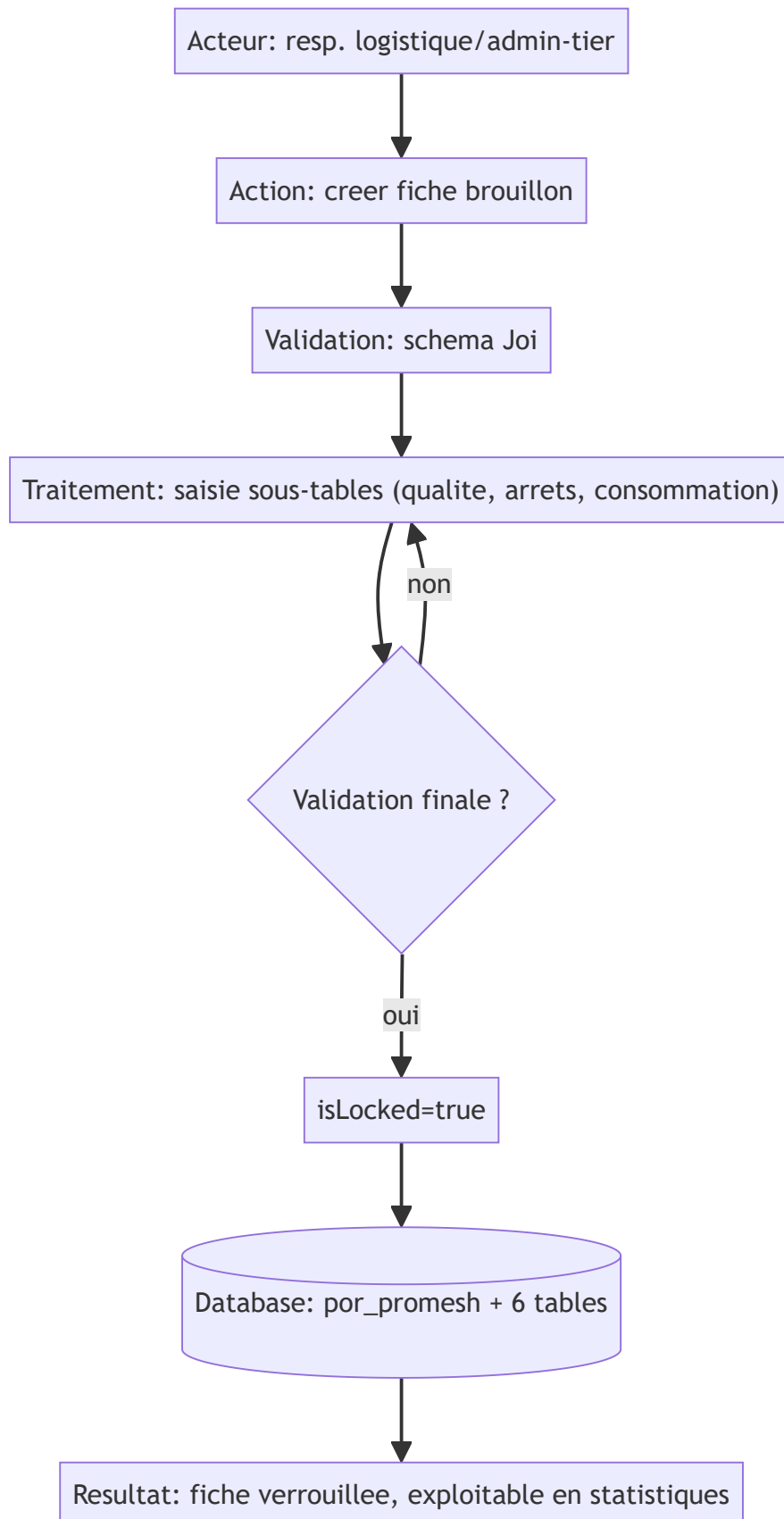
13.3 Demande de maintenance



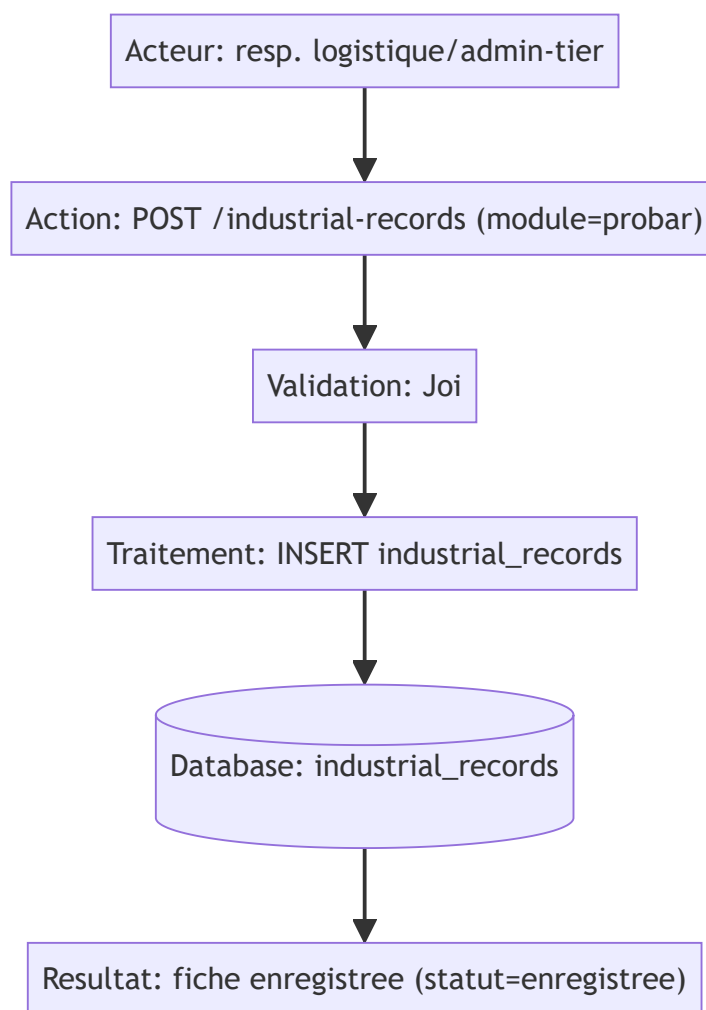
13.4 Demande RH



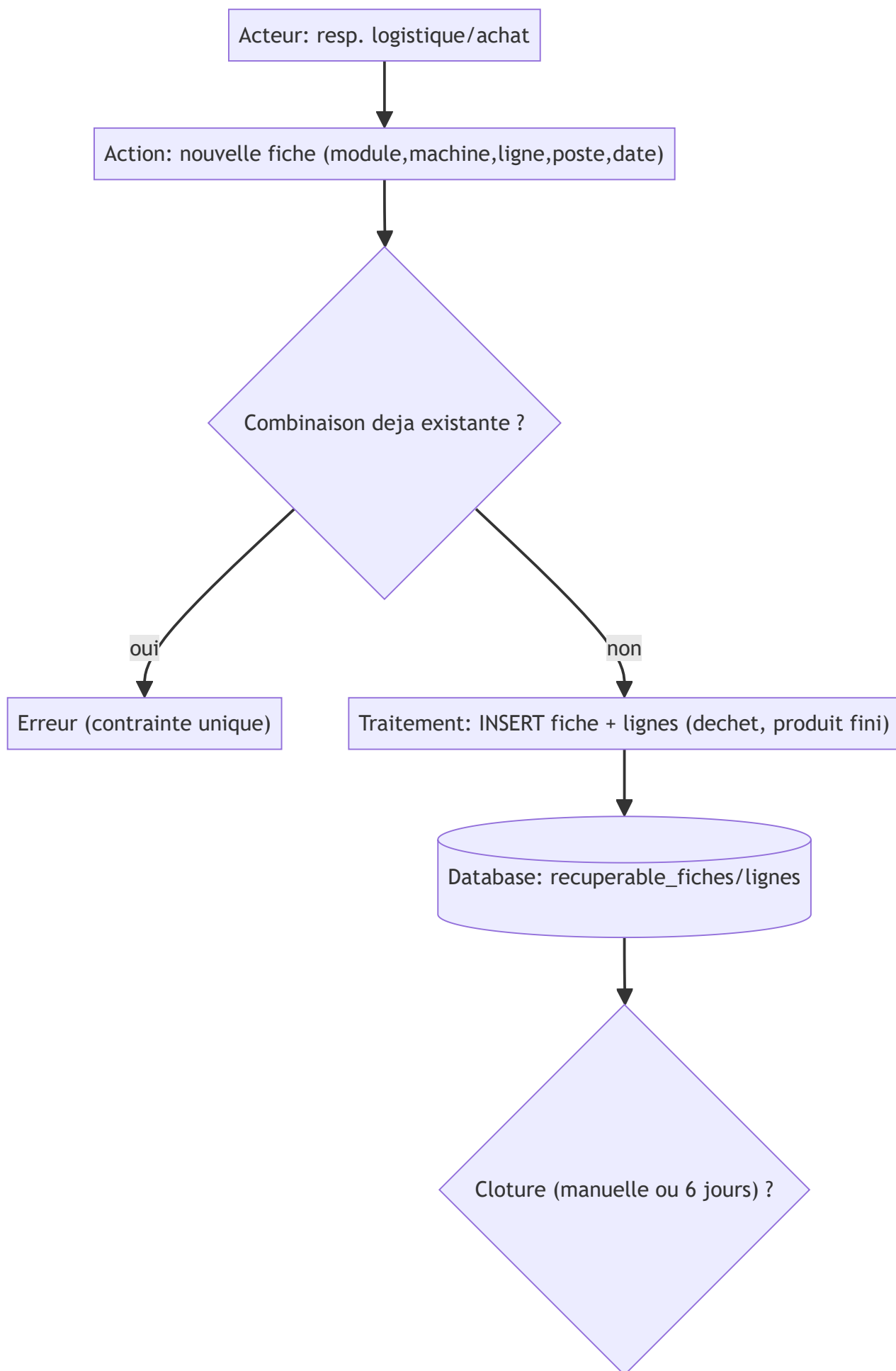
13.5 Production PROMESH (POR PROMESH)

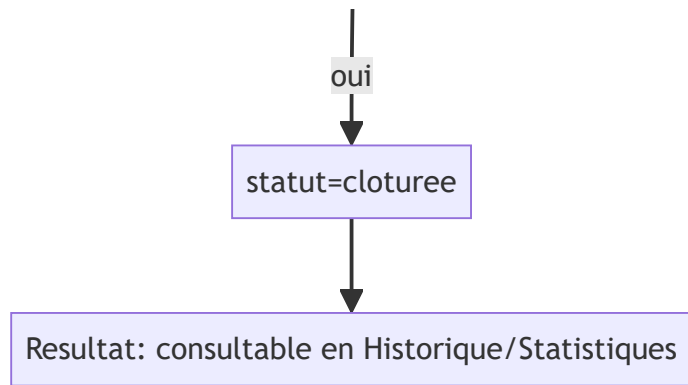


13.6 Production PROBAR (table générique)



13.7 Récupérables





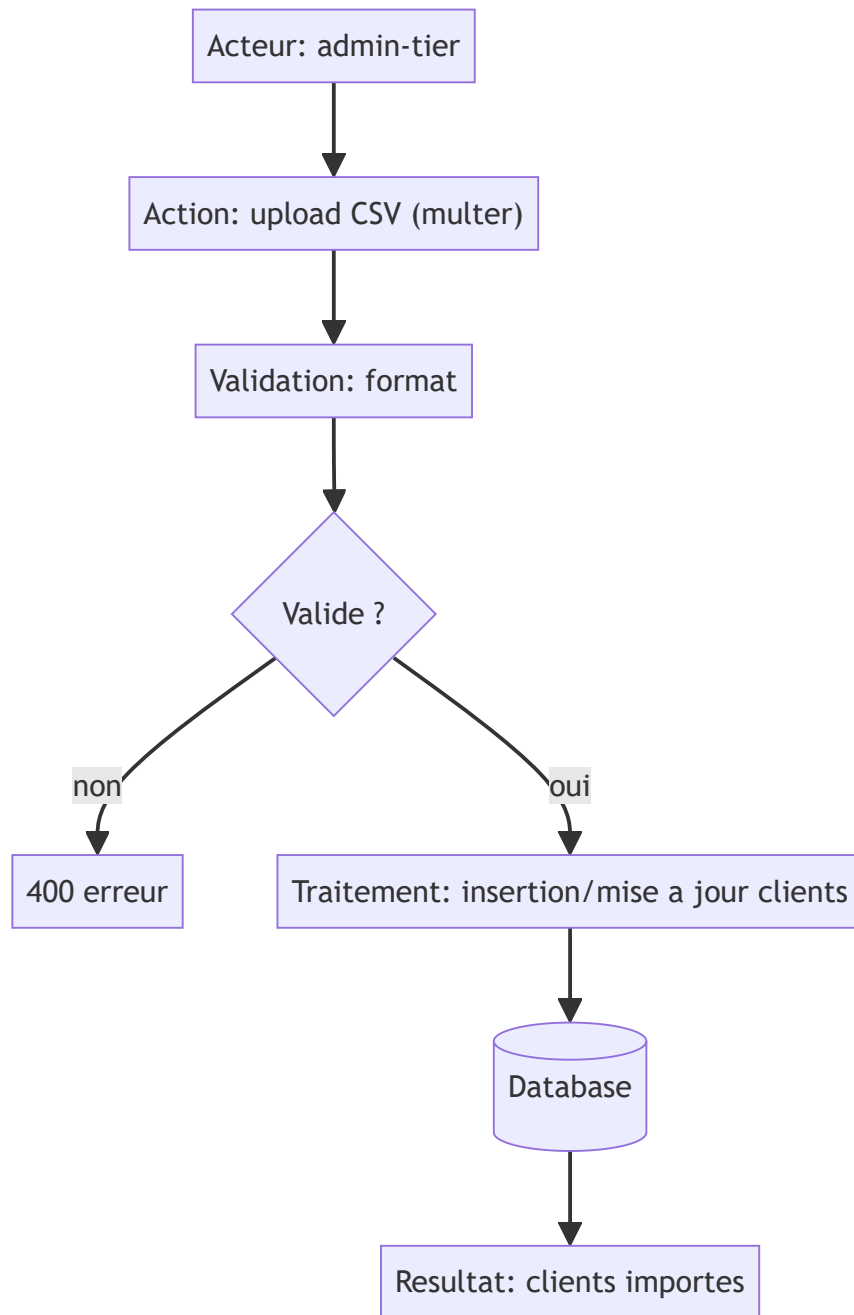
13.8-13.9 Waste / Finished Product (calcul)

Pas de workflow séparé : `dechetKg` et `dechetProduitFinKg` sont deux champs saisis directement sur `RecuperableLigne` (section 13.7) — aucun calcul automatique dérivé n'a été identifié dans le code analysé au-delà de la saisie brute.

13.10 Finance

Non identifié dans les sources analysées. Aucun workflow de facturation/paiement — seul un import de fichier existe (section 3, BF-19), sans traitement métier Facture/Paiement en aval confirmé.

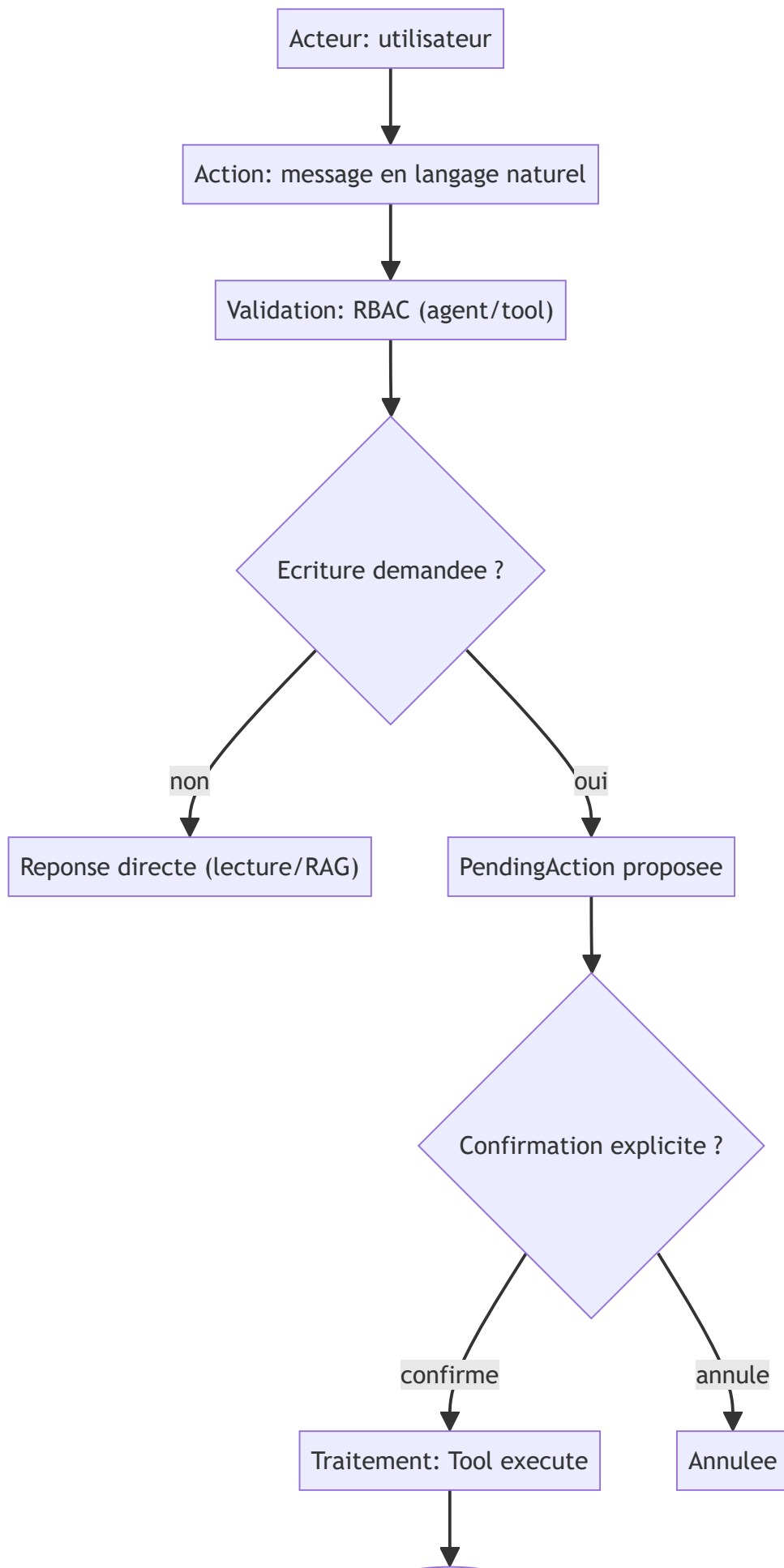
13.11 Import (CSV clients)

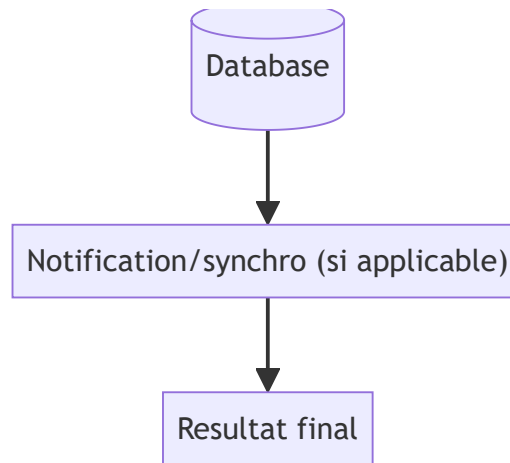


13.12 Validation (générique, illustrée par POR PROMESH)

Voir 13.5 — le pattern de validation le plus riche du projet.

13.13 Copilot IA (action avec confirmation)





14. Questions du jury — niveau métier

30 questions minimum, catégories A-G (compréhension du besoin, problématique, besoins spécifiques, besoins génériques, utilisateurs, processus métier, workflows).

Q1. Quel problème métier votre CRM résout-il ?

COURTE: centraliser des processus dispersés (projets, RH, maintenance, production) avec un RBAC unifié. **DÉTAILLÉE:** avant centralisation, chaque processus avait potentiellement sa propre logique de permission ; le code confirme 17+ modules avec un RBAC transverse partagé (isAuthorized). **POINT À RETENIR:** je ne prétends jamais connaître le « avant » réel, seulement ce que le code centralise aujourd'hui.

Q2. Qui utilise ce système ?

COURTE: 7 rôles réels de l'ENUM Sequelizeize. **DÉTAILLÉE:** user, commercial, accueil, admin, superadmin, superadmin2, responsable_logistique_achat — confirmés dans `User.js:15-20`. **POINT À RETENIR:** ne jamais citer un rôle non présent dans ce code (ex. « finance »).

Q3. Quels départements sont couverts ?

COURTE: Commercial, RH, Maintenance, Logistique/Achat, Administration. **DÉTAILLÉE:** déduit des modules et des comptes désignés par email (gestionnaire RH, maintenance). **POINT À RETENIR:** aucun département Finance identifié.

Q4. Qu'est-ce qui rend ce projet différent d'un CRM générique ?

COURTE: le sous-système industriel (POR PROMESH, IndustrialRecord, Récupérables). **DÉTAILLÉE:** un CRM générique n'a pas de notion de fiche de production verrouillée ni de suivi déchet/produit fini par poste. **POINT À RETENIR:** c'est mon meilleur argument de spécificité (section 3).

Q5. Pourquoi POR PROMESH a-t-il un schéma différent de PROBAR ?

COURTE: POR PROMESH est normalisé (7 tables), PROBAR partage une table générique avec Mélange et Maintenance. **DÉTAILLÉE:** confirmé par le commentaire du modèle IndustrialRecord lui-même. **POINT À RETENIR:** j'assume cette hétérogénéité, je ne la cache pas — c'est un constat honnête, pas une erreur cachée.

Q6. Qu'est-ce que le rôle « responsable_logistique_achat » ?

COURTE: seul rôle explicitement restreint à un périmètre HTTP fixe. **DÉTAILLÉE:** `moduleAccessGuard.js` limite ce rôle à POR PROMESH, Industrial Records, Maintenance, HR, Récupérables. **POINT À RETENIR:** un rôle métier réel, pas un rôle générique CRM.

Q7. Comment une demande RH est-elle traitée ?

COURTE: soumission ouverte, décision réservée au gestionnaire RH désigné. **DÉTAILLÉE:** cycle en_attente→acceptee/refusee→traitee. **POINT À RETENIR:** deux acceptions de « maintenance » existent dans ce projet — ne pas les confondre.

Q8. Qu'est-ce que le verrouillage POR PROMESH ?

COURTE: une fiche validée devient définitivement non modifiable. **DÉTAILLÉE:** champ `isLocked`, vérifié côté service avant toute écriture. **POINT À RETENIR:** exemple parfait de règle métier spécifique.

Q9. Comment le Copilot IA s'intègre-t-il au métier ?

COURTE: comme un point d'entrée conversationnel supplémentaire, jamais un contournement des règles existantes. **DÉTAILLÉE:** chaque agent délègue aux mêmes services/permissions que l'interface classique. **POINT À RETENIR:** l'IA n'invente pas de nouvelle logique métier.

Q10. Quels processus étaient manuels avant ce projet ?

COURTE: non identifié dans les sources analysées. **DÉTAILLÉE:** aucune trace documentaire d'un état antérieur n'existe dans le dépôt. **POINT À RETENIR:** réponse honnête à privilégier plutôt qu'une supposition.

Q11. Pourquoi une relance commerciale nécessite-t-elle une confirmation ?

COURTE: pour qu'un LLM ne déclenche jamais une écriture réelle sans supervision humaine. **DÉTAILLÉE:** cycle propose→confirme→exécute, `consume()` atomique empêche la double exécution. **POINT À RETENIR:** lié à un incident réel observé (hallucination de tool-calling).

Q12. Qu'est-ce qu'une fiche Récupérables ?

COURTE: le suivi du déchet et du produit fini récupéré, par machine/ligne/poste/date. **DÉTAILLÉE:** une seule fiche par combinaison exacte (contrainte unique). **POINT À RETENIR:** Waste/Finished Product sont deux champs numériques, pas des entités séparées.

Q13. Qui peut approuver un archivage de projet ?

COURTE: un seul compte désigné par email (root admin). **DÉTAILLÉE:** `requireRootAdmin`, pas un rôle ENUM générique. **POINT À RETENIR:** modèle hybride rôle-ou-email, propre à ce projet.

Q14. Le CRM gère-t-il la facturation ?

COURTE: non. **DÉTAILLÉE:** aucun modèle Invoice/Payment/Shipment identifié ; seul un import de fichier « factures » existe côté clients. **POINT À RETENIR:** réponse assumée, pas contournée.

Q15. Comment un commercial suit-il ses contacts ?

COURTE: via CommercialContact, avec statut, pipeline de vente et historique. **DÉTAILLÉE:** KPI personnel réservé au rôle commercial/admin-tier. **POINT À RETENIR:** ownership vérifiée en base pour toute écriture.

Q16. Quels sont les 3 modules les plus utilisés selon vous ?

COURTE: Projects, Commercial Contacts, Copilot IA. **DÉTAILLÉE:** jugement basé sur le volume de code (endpoints, agents) plutôt qu'une mesure d'usage réelle non disponible. **POINT À RETENIR:** préciser que c'est une estimation, pas une mesure.

Q17. Pourquoi un utilisateur « accueil » n'a-t-il pas accès au Copilot ?

COURTE: décision produit explicite, confirmée par un test dédié. **DÉTAILLÉE:** `AI_COPILOT_ROLES_ALLOWED` exclut ce rôle. **POINT À RETENIR:** pas un oubli, un choix testé.

Q18. Quelle est la différence entre Maintenance Requests et Industrial Records (maintenance) ?

COURTE: le premier est une demande interne (panne d'équipement bureau/site) ; le second est une fiche de maintenance d'atelier industriel. **DÉTAILLÉE:** deux tables distinctes, deux permissions distinctes. **POINT À RETENIR:** question piège classique — ne pas les confondre à l'oral.

Q19. Comment un projet passe-t-il d'une étape à l'autre ?

COURTE: déplacement kanban, réservé au propriétaire ou à un admin-tier. **DÉTAILLÉE:** `requireOwnerOrAdmin`, invalidation de cache kanban. **POINT À RETENIR:** pas de workflow d'approbation à cette étape (contrairement à RH/Maintenance).

Q20. Le rôle « commercial » peut-il voir les projets d'un autre commercial ?

COURTE: seulement s'il n'est pas admin-tier, il est scopé à ses propres données. **DÉTAILLÉE:** `scopeInputForActingUser` force le périmètre. **POINT À RETENIR:** un incident réel a existé avant ce correctif (fuite de portée) — je peux le citer honnêtement.

Q21. Comment le système notifie-t-il un utilisateur ?

COURTE: Socket.IO en temps réel + persistance en base. **DÉTAILLÉE:** notification créée avant émission, jamais perdue même hors ligne. **POINT À RETENIR:** deux canaux complémentaires, pas redondants.

Q22. Comment le Copilot sait-il qu'une fiche POR PROMESH est verrouillée ?

COURTE: il ne le « sait » pas directement — le Tool appelle le même service que l'interface classique, qui refuse l'écriture. **DÉTAILLÉE:** la règle métier vit dans le service, pas dans l'agent. **POINT À RETENIR:** l'IA ne réimplémente jamais une règle métier.

Q23. Un utilisateur « user » peut-il créer une fiche Récupérables ?

COURTE: non. **DÉTAILLÉE:** réservé admin-tier + responsable_logistique_achat. **POINT À RETENIR:** vérifiable directement dans la matrice section 11.

Q24. Quelle est la valeur ajoutée du Copilot pour un commercial ?

COURTE: programmer une relance en une phrase plutôt qu'une séquence d'écrans. **DÉTAILLÉE:** extraction de créneaux/contact/commercial déterministe, jamais laissée au LLM. **POINT À RETENIR:** gain de manipulation, pas un remplacement du contrôle métier.

Q25. Comment gérez-vous une modification concurrente d'un même projet ?

COURTE: non identifié de mécanisme explicite de verrouillage optimiste dans le code analysé. **DÉTAILLÉE:** dernière écriture gagne (comportement par défaut Sequelize). **POINT À RETENIR:** une limite honnête à assumer si le jury insiste.

Q26. Le CRM gère-t-il plusieurs entreprises (multi-tenant) ?

COURTE: non identifié. **DÉTAILLÉE:** aucune colonne `tenantId` / `companyId` globale contraignant l'isolation des données n'a été trouvée au niveau applicatif transverse. **POINT À RETENIR:** ne jamais affirmer un multi-tenant non vérifié.

Q27. Pourquoi le Mélange partage-t-il la table d'Industrial Records ?

COURTE: choix de modélisation léger pour 3 postes de travail voisins. **DÉTAILLÉE:** champ JSONB `mélangeData` pour les données spécifiques non structurées. **POINT À RETENIR:** compromis assumé entre rapidité de développement et normalisation stricte.

Q28. Qu'est-ce qu'une « Archive Request » ?

COURTE: une demande d'archivage ou de désarchivage d'un projet, soumise à approbation. **DÉTAILLÉE:** statut pending/approved/rejected, décision réservée au root admin. **POINT À RETENIR:** pas une suppression — un archivage réversible.

Q29. Quels workflows impliquent une notification email ?

COURTE: reset de mot de passe, rappels de relance/action (cron), OTP MFA (si branché). **DÉTAILLÉE:** file d'attente avec retry, jamais d'envoi perdu. **POINT À RETENIR:** distinguer email transactionnel (queue) et notification applicative (Socket.IO + DB).

Q30. Quel est le processus le plus critique du point de vue métier ?

COURTE: le verrouillage POR PROMESH, car il garantit l'intégrité des données de production servant aux statistiques. **DÉTAILLÉE:** c'est le seul invariant métier vérifié au niveau service (pas seulement en base). **POINT À RETENIR:** bon exemple à mettre en avant si le jury demande « la règle métier la plus importante ».

15. Questions du jury — architecture

Q1. Pourquoi séparer frontend et backend ?

COURTE: pour cibler 6 plateformes (web, mobile, desktop) avec une seule API. **TECHNIQUE:** Flutter consomme une API REST/WebSocket via Dio, aucune logique métier côté client. **JUSTIFICATION:** dossiers android/ios/web/windows/macos/linux tous réels.

Q2. Pourquoi Node.js/Express pour le backend ?

COURTE: écosystème mature pour une API REST + intégration Socket.IO + kernel IA JavaScript homogène. **TECHNIQUE:** Express 5, tout le kernel IA (LangChain) est également en JavaScript — un seul langage backend. **JUSTIFICATION:** package.json, src/app.js.

Q3. Pourquoi Flutter côté frontend ?

COURTE: un seul code source pour 6 plateformes. **TECHNIQUE:** Dart, GoRouter, GetX. **JUSTIFICATION:** dossiers de plateforme natifs tous présents.

Q4. Pourquoi trois strates de routage différentes coexistent-elles ?

COURTE: évolution progressive du projet, pas un choix initial. **TECHNIQUE:** routes legacy inline, controller simple, modules en couches (Joi+repository) pour les développements récents. **JUSTIFICATION:** confirmé par lecture directe + audit préexistant.

Q5. Pourquoi ne pas avoir refactorisé tout le legacy vers le pattern en couches ?

COURTE: coût de refactor élevé pour un code déjà fonctionnel et testé. **TECHNIQUE:** risque de régression sur ~55 endpoints dans un seul fichier (projects.routes.js). **JUSTIFICATION:** dette technique documentée (section 18), pas ignorée.

Q6. Pourquoi utiliser un Repository ?

COURTE: découpler la logique métier de l'accès aux données. **TECHNIQUE:** présent uniquement dans les modules récents (src/modules/*/repositories/). **JUSTIFICATION:** project.repository.js.

Q7. Pourquoi utiliser des middlewares ?

COURTE: factoriser l'authentification/autorisation avant d'atteindre la logique métier. **TECHNIQUE:** 17 fichiers, chacun avec une responsabilité unique. **JUSTIFICATION:** src/middleware/.

Q8. Comment circule une requête HTTP typique ?

COURTE: helmet→compression→json→cors→moduleAccessGuard→authRequired→middleware spécialisé→route/service→Sequelize→réponse. **TECHNIQUE:** ordre exact confirmé dans app.js. **JUSTIFICATION:** diagramme section 2 du document technique.

Q9. Où se fait la validation des entrées ?

COURTE: Joi dans les modules récents, validation manuelle inline dans le legacy. **TECHNIQUE:** deux approches coexistent, jamais unifiées. **JUSTIFICATION:** project.validator.js vs fonctions inline dans projects.routes.js.

Q10. Où se fait l'autorisation ?

COURTE: middleware avant la route, parfois vérification inline dans le service (ownership). **TECHNIQUE:** isAuthorized() unique, partagée entre CRM et IA. **JUSTIFICATION:** utils/authorization.js.

Q11. Comment gérez-vous les erreurs ?

COURTE: localement, par try/catch dans chaque route — pas de middleware d'erreur centralisé. **TECHNIQUE:** confirmé par recherche exhaustive (aucun middleware 4-arguments). **JUSTIFICATION:** limite assumée, recommandation formulée (section 24 du document technique).

Q12. Comment assurez-vous la maintenabilité ?

COURTE: pattern en couches pour les modules récents, tests Jest sur le kernel IA. **TECHNIQUE:** ~92% de couverture IA (audit préexistant). **JUSTIFICATION:** hétérogène — pas une maintenabilité uniforme sur tout le projet.

Q13. Comment assurez-vous la scalabilité ?

COURTE: partiellement — le pool Sequelize et Express scalent, mais l'état IA (PendingAction, mémoire conversation) est en mémoire process, non partagé. **TECHNIQUE:** limite documentée explicitement. **JUSTIFICATION:** ne jamais prétendre une scalabilité horizontale non vérifiée.

Q14. Que se passe-t-il si le backend tombe ?

COURTE: toutes les requêtes échouent, l'état IA en mémoire (conversations, PendingActions) est perdu. **TECHNIQUE:** aucune haute disponibilité multi-instance identifiée. **JUSTIFICATION:** limite assumée, pas cachée.

Q15. Que se passe-t-il si la base de données tombe ?

COURTE: toutes les écritures/lectures échouent ; le pool Sequelize retente les connexions selon sa configuration. **TECHNIQUE:** pool {max:10,min:0,acquire:30000,idle:10000}. **JUSTIFICATION:** src/db.js.

Q16. Pourquoi le kernel IA est-il séparé du reste du backend ?

COURTE: Ports & Adapters — dépendance unidirectionnelle, jamais l'inverse. **TECHNIQUE:** vérifié mécaniquement par des tests qui lisent le code source. **JUSTIFICATION:** tests ai.*.policy.test.js.

Q17. Pourquoi ne pas avoir mis toute la logique dans le frontend ?

COURTE: sécurité — toute règle métier vérifiée côté client serait contournable. **TECHNIQUE:** RBAC, ownership, validation systématiquement côté serveur. **JUSTIFICATION:** confirmé par les audits sécurité.

Q18. Pourquoi deux mécanismes JWT parallèles existent-ils (authRequired / verifyToken.js) ?

COURTE: évolution historique non unifiée, un seul consommateur (metabase.js) utilise l'ancien. **TECHNIQUE:** deux variables d'environnement différentes. **JUSTIFICATION:** dette technique identifiée (TD-04).

Q19. Pourquoi GetX plutôt que Provider partout côté Flutter ?

COURTE: GetX est dominant (~130 fichiers), Provider reste en périphérie pour l'auth globale et la langue. **TECHNIQUE:** les deux coexistent réellement — pas un choix pur. **JUSTIFICATION:** comptage par grep confirmé.

Q20. Pourquoi la table Industrial Records est-elle générique plutôt que 3 tables séparées ?

COURTE: 3 postes voisins avec un schéma commun suffisant. **TECHNIQUE:** colonne ENUM `module` discriminante. **JUSTIFICATION:** commentaire du modèle lui-même le documente.

Q21. Comment le Copilot appelle-t-il une fonction backend ?

COURTE: via un protocole JSON structuré interprété par ToolCallingChain, jamais l'API function-calling propriétaire d'un vendeur. **TECHNIQUE:** compatible avec n'importe quel ILLmProvider. **JUSTIFICATION:** toolCallingChain.js.

Q22. Pourquoi ne pas avoir utilisé un ORM différent (Prisma, TypeORM) ?

COURTE: Sequelize était déjà en place, migration complète non justifiée pour ce projet. **TECHNIQUE:** non identifié de comparaison documentée dans le dépôt. **JUSTIFICATION:** réponse honnête, pas de justification technique retrouvée au-delà du choix initial.

Q23. Pourquoi le projet n'a-t-il pas de Docker ?

COURTE: non mis en place à ce stade, confirmé absent. **TECHNIQUE:** déploiement direct via PM2 référencé. **JUSTIFICATION:** dette technique identifiée, recommandation formulée.

Q24. Comment le frontend sait-il qu'un token a expiré ?

COURTE: une erreur 401 déclenche un nettoyage du stockage local côté client. **TECHNIQUE:** pas de refresh automatique ni de redirection immédiate garantie (limite identifiée). **JUSTIFICATION:** ApiClient.dio interceptor onError.

Q25. Pourquoi certains modules ont-ils un controller et d'autres non ?

COURTE: selon l'époque et la complexité du module au moment de son développement. **TECHNIQUE:** architect/company/engineer ont un controller simple ; projects legacy n'en a pas. **JUSTIFICATION:** confirmé par lecture directe.

Q26. Comment évitez-vous qu'un module casse un autre module ?

COURTE: séparation par domaine (dossiers dédiés), mais pas de garantie formelle contre le couplage. **TECHNIQUE:** un couplage réel existe (kanban/project/pipelineStage services). **JUSTIFICATION:** constat honnête de l'audit préexistant, pas caché.

Q27. Pourquoi utiliser Socket.IO plutôt qu'un simple polling ?

COURTE: notifications en temps réel sans requêtes répétées. **TECHNIQUE:** `emitToUser()` ciblé par utilisateur. **JUSTIFICATION:** `src/socket.js`.

Q28. Le projet applique-t-il vraiment le Domain-Driven Design ?

COURTE: non, pas formellement. **TECHNIQUE:** pas d'agrégats/value objects explicites ; contextes bornés observés de fait, pas déclarés.

JUSTIFICATION: analyse honnête section 5 du document complet — ne jamais sur-vendre le DDD.

Q29. Comment le Copilot reste-t-il cohérent avec l'architecture CRM existante ?

COURTE: il ne réimplémente jamais une logique métier — chaque Tool délègue à un service CRM déjà existant. **TECHNIQUE:** vérifié par un test qui interdit l'accès direct à Sequelize depuis un Tool. **JUSTIFICATION:** `ai.mcp.tools.policy.test.js`.

Q30. Quel est le principal risque architectural aujourd'hui ?

COURTE: l'absence de gestion d'erreur centralisée et l'hétérogénéité des trois strates de routage. **TECHNIQUE:** impacte la cohérence des réponses API et la sécurité (fuite de stack trace). **JUSTIFICATION:** section 18, TD-09.

16. Questions du jury — base de données

Q1. Pourquoi PostgreSQL ?

COURTE: base relationnelle robuste, supportant JSONB et ENUM natifs, déjà en place. **TECHNIQUE:** utilisé via Sequelize (pg/pg-hstore). **JUSTIFICATION:** src/db.js, migrations.

Q2. Pourquoi utiliser un ORM plutôt que du SQL brut ?

COURTE: productivité et portabilité des modèles. **TECHNIQUE:** Sequelize, sauf exceptions volontaires (SQL brut dans commercialContactRelance.service.js pour rester mockable). **JUSTIFICATION:** commentaire du code lui-même.

Q3. Pourquoi ces relations entre tables ?

COURTE: elles reflètent les dépendances métier réelles (Project→PipelineStage, PorPromesh→ses 6 sous-tables). **TECHNIQUE:** déclarées dans associations.js. **JUSTIFICATION:** lecture exhaustive du fichier.

Q4. Pourquoi utiliser des clés étrangères ?

COURTE: garantir l'intégrité référentielle et définir le comportement de suppression (CASCADE/SET NULL). **TECHNIQUE:** ex. HrRequestActivity.requestId → CASCADE. **JUSTIFICATION:** modèles Sequelize.

Q5. Comment garantisiez-vous l'intégrité des données ?

COURTE: contraintes FK + unique + ENUM au niveau base, complétées par des vérifications métier au niveau service. **TECHNIQUE:** ex. unicité (module,machine,ligne,poste,date). **JUSTIFICATION:** RecuperableFiche.

Q6. Comment évitez-vous les doublons ?

COURTE: contraintes uniques (simples ou composites). **TECHNIQUE:** unique(email) sur users, unique(name where deletedAt null) sur pipeline_stages. **JUSTIFICATION:** modèles lus intégralement.

Q7. Comment gérez-vous les migrations ?

COURTE: exclusivement via sequelize-cli, jamais de sync automatique. **TECHNIQUE:** confirmé par un commentaire explicite du code. **JUSTIFICATION:** app.js, package.json (db:migrate).

Q8. Pourquoi utiliser des index ?

COURTE: accélérer les requêtes fréquentes (filtrage par statut, date). **TECHNIQUE:** ex. index composite sur industrial_records(module,dateFiche,createdAt). **JUSTIFICATION:** ajoutés suite à un incident de performance réel documenté.

Q9. Comment gérez-vous les transactions ?

COURTE: utilisées pour les écritures multi-tables (ex. fiche+lignes Récupérables). **TECHNIQUE:** pas systématiques sur tout le legacy (dette identifiée). **JUSTIFICATION:** project.service.js::moveStage() cité par l'audit préexistant comme bon exemple.

Q10. Pourquoi utiliser des ENUM plutôt que des tables de référence ?

COURTE: simplicité pour des ensembles de valeurs fixes et rarement modifiés (statuts, rôles). **TECHNIQUE:** ex. User.role, HrRequest.statut. **JUSTIFICATION:** confirmé dans tous les modèles à statut.

Q11. Comment éviter la perte de données lors d'une suppression ?

COURTE: CASCADE volontaire pour les données dépendantes, SET NULL pour préserver l'historique. **TECHNIQUE:** ex. PipelineStage en soft-delete (paranoid: true). **JUSTIFICATION:** deletedAt.

Q12. Comment sauvegardez-vous la base ?

COURTE: non identifié dans le code source analysé — responsabilité infrastructure hors dépôt. **TECHNIQUE:** aucun script de backup trouvé. **JUSTIFICATION:** réponse honnête.

Q13. Que se passe-t-il si une migration échoue en production ?

COURTE: non identifié de mécanisme de rollback automatique au-delà de db:migrate:undo. **TECHNIQUE:** le CI actuel tolère l'échec de migration (continue-on-error) — dette suivie séparément. **JUSTIFICATION:** backend-ci.yml.

Q14. Pourquoi certaines tables n'ont-elles pas de champ de statut ?

COURTE: parce qu'elles sont des logs append-only (ProjectActivity) ou des données sans cycle de vie (Company, ProjectDevis).

TECHNIQUE: confirmé exhaustivement (section 10 du document complet). **JUSTIFICATION:** jamais inventé de statut absent.

Q15. Pourquoi le kernel IA n'a-t-il aucune table dédiée ?

COURTE: choix assumé — l'état conversationnel est volatile par conception. **TECHNIQUE:** confirmé par recherche exhaustive et par un commentaire du code. **JUSTIFICATION:** aiCopilotBridge.service.js.

Q16. Comment le schéma évolue-t-il dans le temps ?

COURTE: par migrations versionnées et datées. **TECHNIQUE:** ex. ajout de superadmin2 au rôle, ajout de copilot_meeting à l'ENUM CalendarEventSync. **JUSTIFICATION:** src/migrations/.

Q17. Pourquoi IndustrialRecord n'a-t-il pas 3 tables séparées comme PorPromesh ?

COURTE: choix de modélisation plus léger pour un besoin jugé plus simple. **TECHNIQUE:** colonne JSONB pour l'extensibilité (Mélange). **JUSTIFICATION:** commentaire du modèle.

Q18. Comment vérifiez-vous qu'une relance appartient bien à l'utilisateur ?

COURTE: comparaison explicite en base (contact.createdBy==actingUser.id) avant toute écriture. **TECHNIQUE:** pas seulement une vérification applicative — une requête dédiée. **JUSTIFICATION:** commercialContactRelance.service.js.

Q19. Y a-t-il des incohérences dans le modèle de données ?

COURTE: oui, une identifiée — ProjectComment déclare authorId dans un sens et userId dans l'autre. **TECHNIQUE:** associations.js:162,164. **JUSTIFICATION:** assumé dans la section problèmes techniques, pas caché.

Q20. Comment gérez-vous les timestamps ?

COURTE: createdAt/updatedAt automatiques par Sequelize, désactivés (updatedAt:false) pour les logs append-only. **TECHNIQUE:** ex. ProjectActivity. **JUSTIFICATION:** confirmé modèle par modèle.

Q21. Pourquoi utiliser des UUID plutôt que des identifiants auto-incrémentés ?

COURTE: éviter l'énumération d'identifiants séquentiels, cohérence entre systèmes distribués. **TECHNIQUE:** tous les id primaires sont UUID (sauf ticketNo, volontairement auto-incrémenté pour lisibilité humaine). **JUSTIFICATION:** HrRequest.ticketNo.

Q22. Comment le pool de connexions est-il configuré ?

COURTE: max 10, min 0, acquire 30000ms, idle 10000ms. **TECHNIQUE:** src/db.js. **JUSTIFICATION:** lu directement.

Q23. Comment structurez-vous les données de production (PROBAR/Mélange) ?

COURTE: une table generique avec discriminant ENUM. **TECHNIQUE:** industrial_records(module). **JUSTIFICATION:** IndustrialRecord.js.

Q24. Que se passe-t-il si deux fiches Récupérables identiques sont soumises simultanément ?

COURTE: la contrainte unique en base rejette la seconde. **TECHNIQUE:** erreur d'insertion gérée côté service. **JUSTIFICATION:** contrainte confirmée dans le modèle.

Q25. Le modèle de données est-il documenté formellement (ERD officiel) ?

COURTE: pas nativement dans le dépôt — cette documentation d'audit produit l'ERD reconstitué. **TECHNIQUE:** aucun outil de schéma généré automatiquement identifié. **JUSTIFICATION:** réponse honnête.

17. Questions du jury — sécurité

Q1. Comment fonctionne l'authentification ?

COURTE: JWT access(15min)+refresh(7j, cookie httpOnly), bcrypt pour les mots de passe. **TECHNIQUE:** authRequired décode et pose req.user. **JUSTIFICATION:** auth.middleware.js.

Q2. Pourquoi JWT plutôt que des sessions serveur ?

COURTE: sans état côté serveur, adapté à une API consommée par plusieurs plateformes clientes. **TECHNIQUE:** deux secrets distincts (access/refresh). **JUSTIFICATION:** utils/tokens.js.

Q3. Comment fonctionne le RBAC ?

COURTE: une fonction unique isAuthorized(), partagée par tout le backend et le Copilot. **TECHNIQUE:** modèle hybride rôle-ou-email. **JUSTIFICATION:** utils/authorization.js.

Q4. Comment empêchez-vous un utilisateur d'accéder à un module interdit ?

COURTE: middleware avant la route + vérification RBAC dans le service. **TECHNIQUE:** moduleAccessGuard (global) + middlewares spécialisés. **JUSTIFICATION:** app.js:126.

Q5. Différence entre authentification et autorisation ?

COURTE: authentification = qui vous êtes (JWT vérifié) ; autorisation = ce que vous avez le droit de faire (RBAC). **TECHNIQUE:** deux étapes distinctes dans le pipeline (authRequired puis middleware spécialisé). **JUSTIFICATION:** ordre exact dans app.js.

Q6. Pourquoi un utilisateur peut-il recevoir 403 ?

COURTE: rôle non autorisé pour ce module/action, ou non-ownership. **TECHNIQUE:** moduleAccessGuard, kpiRoleGuard, requireRootAdmin/HrReviewer/MaintenanceManager. **JUSTIFICATION:** section 12 du document technique.

Q7. Où sont contrôlées les permissions ?

COURTE: à plusieurs niveaux — middleware de route, vérification inline dans le service, et pour l'IA, PermissionManager au niveau du Tool. **TECHNIQUE:** jamais un seul point de contrôle. **JUSTIFICATION:** défense en profondeur vérifiée.

Q8. Comment protégez-vous les APIs ?

COURTE: authRequired systématique + validation Joi/manuelle + rate limiting sur les routes sensibles. **TECHNIQUE:** pas systématique partout (dette identifiée). **JUSTIFICATION:** section 14 du document complet.

Q9. Comment protégez-vous les fichiers uploadés ?

COURTE: partiellement — multer filtre extension/taille, mais la route /uploads est servie sans authentification. **TECHNIQUE:** faiblesse identifiée et documentée. **JUSTIFICATION:** TD-02, honnêteté requise si le jury insiste.

Q10. Comment gérez-vous les secrets ?

COURTE: variables d'environnement, jamais committées en clair dans le code applicatif ; erreurs LLM jamais porteuses de la clé API. **TECHNIQUE:** [SECRET] dans toute documentation produite. **JUSTIFICATION:** providers/errors.js.

Q11. Comment protégez-vous les mots de passe ?

COURTE: hachage bcrypt, jamais stockés en clair. **TECHNIQUE:** reset password via token SHA-256 haché, TTL 15 min. **JUSTIFICATION:** auth.routes.js.

Q12. Comment fonctionne le MFA ?

COURTE: OTP par email, appareils de confiance, mais jamais déclenché par la connexion réelle aujourd'hui. **TECHNIQUE:** infrastructure complète et testée, débranchée du flux signifié. **JUSTIFICATION:** honnêteté totale requise — c'est une limite documentée, pas une fonctionnalité vendue comme active.

Q13. Comment gérez-vous les tokens expirés ?

COURTE: 401 renvoyé, le client nettoie son stockage local. **TECHNIQUE:** pas de refresh automatique transparent identifié côté Flutter. **JUSTIFICATION:** `api_client.dart` `onError`.

Q14. Comment évitez-vous l'injection SQL ?

COURTE: usage systématique de l'ORM (requêtes paramétrées) ou de requêtes préparées explicites. **TECHNIQUE:** aucune concaténation de requête brute identifiée dans le code analysé. **JUSTIFICATION:** audit préexistant confirme l'absence d'injection SQL trouvée.

Q15. Comment gérez-vous le CORS ?

COURTE: whitelist d'origines en dur. **TECHNIQUE:** `credentials:true`, méthodes limitées. **JUSTIFICATION:** `app.js:111-123`.

Q16. Comment protégez-vous contre le rate-limiting abusif ?

COURTE: `express-rate-limit` sur `auth/geocode/import`, pas systématique ailleurs. **TECHNIQUE:** dette identifiée pour les exports coûteux. **JUSTIFICATION:** section 18 du document complet.

Q17. Le Copilot peut-il contourner le RBAC du CRM ?

COURTE: non, structurellement. **TECHNIQUE:** `isAuthorized()` unique et partagée, jamais réimplémentée par l'IA. **JUSTIFICATION:** audit sécurité/RBAC IA dédié.

Q18. Qu'est-ce que la fuite de stack trace ?

COURTE: une réponse d'erreur 500 qui expose la stack technique brute au client. **TECHNIQUE:** confirmé dans `commercial_contacts.routes.js`. **JUSTIFICATION:** risque de divulgation d'information, recommandation formulée.

Q19. Comment auditez-vous les actions sensibles ?

COURTE: journaux d'activité append-only (`ProjectActivity`, `HrRequestActivity`, `MfaAttemptLog`). **TECHNIQUE:** pas un audit trail transverse unifié. **JUSTIFICATION:** par domaine, pas centralisé.

Q20. Comment gérez-vous les rôles désignés par email plutôt que par rôle ENUM ?

COURTE: modèle hybride « rôle OU email », implémenté une seule fois (`isAuthorized`) et réutilisé. **TECHNIQUE:** pas dupliqué à chaque middleware. **JUSTIFICATION:** commentaire du code lui-même.

Q21. Que risque un utilisateur qui appelle directement une API interdite (sans passer par l'UI) ?

COURTE: rien de plus qu'un utilisateur normal — la sécurité n'est jamais basée sur l'UI. **TECHNIQUE:** middleware serveur systématique. **JUSTIFICATION:** RBAC vérifié côté API, pas côté client.

Q22. Comment gérez-vous la confidentialité des données RH ?

COURTE: accès restreint au demandeur et au gestionnaire RH désigné. **TECHNIQUE:** `requireHrReviewer`. **JUSTIFICATION:** `hrRequest.routes.js`.

Q23. Le refresh token peut-il être joué s'il est volé ?

COURTE: oui, potentiellement — pas de rotation ni détection de réutilisation. **TECHNIQUE:** limite documentée (TD-12). **JUSTIFICATION:** honnêteté requise.

Q24. Comment le Copilot empêche-t-il une fuite de données entre utilisateurs ?

COURTE: `scopeInputForActingUser()` force le périmètre, même si le LLM fournit un champ halluciné. **TECHNIQUE:** incident réel corrigé (`list_projects`). **JUSTIFICATION:** audit sécurité IA dédié.

Q25. Quelle est la plus grande vulnérabilité connue aujourd'hui ?

COURTE: la route `/uploads` non authentifiée. **TECHNIQUE:** tout fichier au chemin deviné est accessible sans JWT. **JUSTIFICATION:** TD-02, réponse honnête si demandée en soutenance.

18. Questions du jury sur l'intelligence artificielle

Section la plus importante de ce document. 40 questions, chacune avec réponse 20s / réponse 1min / réponse technique / piège / erreur à éviter.

Q1. Pourquoi avez-vous intégré l'IA dans le CRM ?

20S: pour permettre un accès en langage naturel aux données déjà exposées par le CRM, sans naviguer entre les écrans. **1MIN:** 19 agents spécialisés couvrent projets, commercial, RH, maintenance, production — chacun délègue aux mêmes services que l'interface classique.

TECHNIQUE: orchestrateur (AIManager) → IntentDetector → Agent → Tool MCP → service CRM. **PIÈGE:** « est-ce un gadget ? » **ERREUR À ÉVITER:** prétendre que l'IA remplace l'interface classique — elle la complète.

Q2. Quel problème métier l'IA résout-elle ?

20S: réduire la charge de traduction manuelle d'une intention métier en séquence d'écrans. **1MIN:** ex. programmer une relance nécessite normalement plusieurs écrans ; en une phrase, le Copilot le fait après confirmation. **TECHNIQUE:** PendingAction. **PIÈGE:** « avez-vous mesuré ce gain ? » **ERREUR:** inventer un chiffre de gain de productivité non mesuré.

Q3. Pourquoi un LLM plutôt qu'une recherche SQL classique ?

20S: pour comprendre une formulation libre en français, pas seulement des mots-clés exacts. **1MIN:** l'intent detection reste lexicale et déterministe (pas de LLM) ; le LLM n'intervient qu'après, pour choisir l'outil et rédiger la réponse. **TECHNIQUE:** LexicalIntentRouter (sans LLM) puis ToolCallingChain (avec LLM). **PIÈGE:** « pourquoi pas tout en SQL ? » **ERREUR:** dire que le LLM fait la recherche — c'est le Tool (SQL via Sequelize) qui la fait.

Q4. Quelle différence entre un LLM et votre CRM ?

20S: le LLM génère du texte, le CRM détient la vérité des données. **1MIN:** le LLM ne connaît jamais une donnée métier directement — il doit toujours passer par un Tool qui interroge PostgreSQL. **TECHNIQUE:** aucune donnée métier n'est dans le prompt système, seulement le résultat d'un Tool exécuté. **PIÈGE:** « le LLM connaît-il déjà les données ? » **ERREUR:** répondre oui — c'est faux.

Q5. Pourquoi avez-vous utilisé RAG ?

20S: pour répondre aux questions documentaires (« comment créer un projet ? ») en s'ancrant sur la vraie documentation, jamais sur des données métier. **1MIN:** SourceGuard interdit structurellement au RAG de lire les données CRM. **TECHNIQUE:** allowlist d'extensions + blocklist de chemins métier + anti-symlink. **PIÈGE:** « le RAG peut-il fuiter des données clients ? » **ERREUR:** dire « non, jamais testé » — dire « non, structurellement impossible, vérifié ».

Q6. Pourquoi ne pas simplement envoyer toute la base de données au LLM ?

20S: taille (fenêtre de contexte limitée), coût, et fuite de données sans contrôle RBAC. **1MIN:** un incident réel a montré qu'un résultat non scopé faisait exploser un prompt à 42 115 caractères, causant un timeout. **TECHNIQUE:** troncature à 4000 caractères (MAX_TOOL_RESULT_CHARS). **PIÈGE:** « pourquoi cette limite précise ? » **ERREUR:** ne pas connaître la valeur exacte.

Q7. Qu'est-ce qu'un embedding ?

20S: une représentation numérique (vecteur) d'un texte, permettant de mesurer sa proximité avec un autre texte. **1MIN:** ici, un hachage déterministe (feature hashing signé), pas un modèle neuronal. **TECHNIQUE:** 256 dimensions, aucun appel réseau. **PIÈGE:** « est-ce un vrai modèle d'IA ? » **ERREUR:** dire oui — c'est un algorithme de hachage, assumé comme choix de phase.

Q8. Pourquoi utiliser une vector database ?

20S: en théorie pour chercher rapidement les passages similaires à une question. **1MIN:** en pratique, ce projet n'utilise PAS de vraie base vectorielle — un simple Map() en mémoire, avec recherche cosinus brute-force. **TECHNIQUE:** InMemoryVectorStore, volatile. **PIÈGE:** « quelle techno de vector DB utilisez-vous ? » **ERREUR:** inventer un nom (Pinecone, Weaviate) — répondre honnêtement qu'aucune n'est utilisée.

Q9. Comment fonctionne le retrieval ?

20S: la question est transformée en vecteur, comparée par similarité cosinus à tous les chunks, puis rerankée. **1MIN:** topK=5, candidateK=20, reranking par similarité de Jaccard + facteur de catégorie de document. **TECHNIQUE:** RagRetrievalService. **PIÈGE:** « qu'est-ce qu'un cross-encoder ? » **ERREUR:** prétendre en utiliser un — ce projet n'en a pas.

Q10. Qu'est-ce qu'un chunk ?

20S: un fragment de document découpé pour l'indexation. **1MIN:** 800 caractères, overlap 120, découpage par paragraphes. **TECHNIQUE:** RecursiveTextChunker. **PIÈGE:** « pourquoi ces valeurs précises ? » **ERREUR:** ne pas savoir répondre — ce sont des valeurs de configuration par défaut, pas un choix scientifique documenté.

Q11. Comment choisissez-vous les documents pertinents ?

20S: via une combinaison score vectoriel + score lexical + facteur de catégorie. **1MIN:** un document de la documentation interne du Copilot est pénalisé ($\times 0.3$) pour éviter qu'il pollue les réponses utilisateur. **TECHNIQUE:** categorizePath(), DEFAULT_CATEGORY_FACTORS. **PIÈGE:** demander un exemple concret. **ERREUR:** ne pas avoir l'exemple « comment créer un projet » (score 0.50 vs 0.08) en tête.

Q12. Qu'est-ce qu'un agent IA ?

20S: un composant spécialisé par domaine métier (projets, RH, maintenance...), avec son propre prompt et ses propres outils autorisés. **1MIN:** 19 agents actifs, RBAC propre à chacun. **TECHNIQUE:** BaseAgent, AgentRegistry. **PIÈGE:** « combien d'agents exactement ? » **ERREUR:** se tromper de chiffre (19, pas 20 — un agent existe mais n'est plus câblé).

Q13. Quelle différence entre un chatbot et un agent ?

20S: un chatbot répond en texte ; un agent peut décider d'appeler un outil pour agir ou consulter une donnée réelle. **1MIN:** ici, chaque agent choisit entre RAG (texte) et Tool Calling (donnée réelle). **TECHNIQUE:** BaseAgent_run(). **PIÈGE:** confondre les deux à l'oral. **ERREUR:** dire qu'un agent est « juste un chatbot avec un nom ».

Q14. Pourquoi utiliser des tools ?

20S: pour que le LLM puisse consulter/modifier une donnée réelle plutôt que de l'halluciner. **1MIN:** 76 outils, chacun validé (Joi) et autorisé (PermissionManager) avant exécution. **TECHNIQUE:** MCP. **PIÈGE:** « combien de tools exactement ? » **ERREUR:** citer un chiffre obsolète des commentaires internes (69/73/74) au lieu de 76 (vérifié par exécution).

Q15. Comment le Copilot appelle-t-il une fonction backend ?

20S: le LLM répond en JSON structuré ("action": "call_tool", ...), jamais l'API function-calling propriétaire d'un vendeur. **1MIN:** ce choix rend le mécanisme compatible avec n'importe quel LLM. **TECHNIQUE:** ToolCallingChain. **PIÈGE:** « pourquoi pas l'API OpenAI function-calling ? » **ERREUR:** ne pas savoir justifier ce choix d'indépendance vis-à-vis du vendeur.

Q16. Comment empêchez-vous l'IA d'accéder à des données interdites ?

20S: chaque Tool revérifie le RBAC via la même fonction que le reste du CRM. **1MIN:** isAuthorized() unique, jamais réimplémentée par l'IA. **TECHNIQUE:** PermissionManager délègue à utils/authorization.js. **PIÈGE:** « et si le LLM insiste ? » **ERREUR:** dire que le prompt empêche ça — c'est le code, pas le prompt.

Q17. Est-ce que l'IA contourne le RBAC ?

20S: non, structurellement impossible. **1MIN:** vérifié par un test e2e réel : un rôle non autorisé reçoit PERMISSION_DENIED avant même l'exécution du Tool. **TECHNIQUE:** ai.security.e2e.test.js. **PIÈGE:** demander une démonstration en direct. **ERREUR:** ne pas connaître ce test précis.

Q18. Que se passe-t-il si le LLM invente une réponse ?

20S: pour les agents à requireToolUse, la réponse est rejetée si aucun outil réel n'a été appelé. **1MIN:** incident réel observé sur POR PROMESH (completion=75 tokens, toolCalls:[]) — corrigé. **TECHNIQUE:** steps.length, jamais le texte du modèle. **PIÈGE:** « est-ce généralisé à tous les agents ? » **ERREUR:** dire oui — seuls 3 agents l'ont (Project, Commercial, PorPromesh).

Q19. Comment gérez-vous les hallucinations ?

20S: jamais en faisant confiance au texte du modèle — toujours par l'état réel de l'exécution. **1MIN:** requireToolUse, citations calculées en code, explainability jamais nulle. **TECHNIQUE:** toolCallingChain.js. **PIÈGE:** « avez-vous éliminé toute hallucination ? » **ERREUR:** dire oui — dire « réduite et détectée, pas éliminée à 100% ».

Q20. Comment gérez-vous le Prompt Injection ?

20S: 5 règles regex bilingues, avec un mode par défaut qui délimite le texte suspect sans le supprimer. **1MIN:** testé conceptuellement contre des attaques de paramètre (ownerId=...) — non capturées par ces règles, mais neutralisées ailleurs (JWT, scoping). **TECHNIQUE:** PromptInjectionGuard. **PIÈGE:** « votre garde capture-t-elle tout ? » **ERREUR:** dire oui — c'est un filet, pas une garantie absolue, et je peux le prouver par le test.

Q21. Est-ce que les données CRM sont envoyées au provider LLM ?

20S: uniquement le résultat d'un Tool explicitement exécuté, jamais la base entière. **1MIN:** le provider actif (Ollama) est local — aucune donnée ne sort du serveur. **TECHNIQUE:** 127.0.0.1:11434. **PIÈGE:** « et si vous passiez à OpenAI ? » **ERREUR:** ne pas anticiper que des données transiteraient alors vers un tiers.

Q22. Comment protégez-vous les informations confidentielles ?

20S: RBAC systématique avant tout accès, RAG qui ne lit jamais les données métier. **1MIN:** secretRedaction avant tout log/audit. **TECHNIQUE:** utils/secretRedaction.js. **PIÈGE:** « et les données RH sensibles au Copilot ? » **ERREUR:** ne pas savoir que HrAgent a son propre RBAC.

Q23. Pourquoi ce modèle LLM (qwen3:8b) ?

20S: exécution locale, sans coût d'API. **1MIN:** aucune comparaison qualitative documentée avec d'autres modèles Ollama n'a été trouvée — c'est une limite honnête à assumer. **TECHNIQUE:** .env AI_LLM_PROVIDER=ollama. **PIÈGE:** « pourquoi pas un modèle plus gros ? » **ERREUR:** inventer une justification de performance non documentée.

Q24. Pourquoi ce provider (Ollama) ?

20S: gratuit, local, sans clé API. **1MIN:** OpenAI et Anthropic sont pleinement implémentés et testés mais non actifs — un seul provider actif à la fois, changeable par une variable d'environnement. **TECHNIQUE:** createLlmProvider(). **PIÈGE:** « pouvez-vous changer de provider facilement ? » **ERREUR:** dire non — l'architecture le permet précisément pour ça.

Q25. Que se passe-t-il si le provider LLM est indisponible ?

20S: l'erreur est classifiée et un message générique sûr est renvoyé, jamais un crash. **1MIN:** retry limité à 2 tentatives sur le même provider, aucun fallback automatique vers un autre. **TECHNIQUE:** LlmProviderUnavailableError. **PIÈGE:** « basculez-vous automatiquement vers un autre LLM ? » **ERREUR:** dire oui — ce n'est pas implémenté, à assumer.

Q26. Comment gérez-vous les erreurs LLM ?

20S: classification typée (auth/timeout/indisponible), message toujours générique et sûr côté utilisateur. **1MIN:** jamais l'erreur brute du fournisseur (pourrait contenir des en-têtes sensibles). **TECHNIQUE:** classifyProviderHttpError(). **PIÈGE:** demander un exemple de message affiché. **ERREUR:** ne pas connaître le texte exact des messages fixes.

Q27. Comment mesurez-vous la qualité des réponses ?

20S: un corpus de 500+ scénarios de test valide le routage d'intention et le comportement des agents. **1MIN:** pas d'évaluation automatique de la qualité sémantique de la réponse finale (pas de LLM-juge). **TECHNIQUE:** copilotValidationScenarios.js. **PIÈGE:** « avez-vous un score de qualité ? » **ERREUR:** inventer un chiffre.

Q28. Comment réduire le coût des appels LLM ?

20S: non pertinent directement — le provider actif (Ollama local) n'a pas de coût par appel. **1MIN:** le coût serait pertinent si un provider payant (OpenAI/Anthropic) était activé — non mesuré dans ce projet. **TECHNIQUE:** aucun tracking de coût identifié dans le code. **PIÈGE:** « quel est votre coût mensuel ? » **ERREUR:** inventer un chiffre.

Q29. Comment réduire le nombre de tokens ?

20S: troncature des résultats d'outils (4000 caractères) et plafond de génération (num_predict=768). **1MIN:** ces limites viennent d'un incident réel de performance mesuré. **TECHNIQUE:** MAX_TOOL_RESULT_CHARS. **PIÈGE:** demander la mesure exacte. **ERREUR:** ne pas connaître 42 115 caractères vs 1 759.

Q30. Pourquoi ne pas utiliser uniquement un LLM sans RAG ?

20S: le LLM n'a aucune connaissance de la documentation interne du CRM — il halluciner sans ancrage. **1MIN:** le RAG garantit une réponse fondée sur un texte réel, citable. **TECHNIQUE:** QaChain refuse de répondre hors contexte. **PIÈGE:** « le LLM seul ne suffit-il pas pour des questions simples ? » **ERREUR:** dire oui sans nuance — il inventerait des noms d'écran/champs inexistantes.

Q31. Pourquoi ne pas utiliser uniquement RAG sans LLM ?

20S: le RAG seul renvoie des extraits bruts, pas une réponse reformulée et conversationnelle. **1MIN:** le LLM synthétise/reformule à partir du contexte fourni par le RAG. **TECHNIQUE:** QaChain combine les deux. **PIÈGE:** demander un exemple. **ERREUR:** ne pas pouvoir illustrer la différence entre extrait brut et réponse reformulée.

Q32. Quelle différence entre RAG et fine-tuning ?

20S: le RAG injecte du contexte à chaque requête sans modifier le modèle ; le fine-tuning modifie les poids du modèle lui-même. **1MIN:** le RAG est plus simple à maintenir à jour (changer un document suffit), le fine-tuning nécessite un réentraînement. **TECHNIQUE:** aucun fine-tuning dans ce projet. **PIÈGE:** confondre les deux notions à l'oral. **ERREUR:** dire que le RAG « entraîne » le modèle — faux.

Q33. Pourquoi ne pas fine-tuner votre propre modèle ?

20S: coût, complexité d'infrastructure, et corpus documentaire trop petit pour un fine-tuning pertinent. **1MIN:** non tenté ni documenté dans ce projet — réponse honnête. **TECHNIQUE:** aucune trace de fine-tuning dans le code. **PIÈGE:** « l'avez-vous envisagé ? » **ERREUR:** inventer une réflexion non documentée.

Q34. Où se trouve le contexte utilisateur ?

20S: dans l'objet `actingUser`, dérivé du JWT, jamais du texte du message. **1MIN:** traduit depuis `{sub,email,role}` du JWT vers `{id,email,role}` au niveau du Bridge. **TECHNIQUE:** `toActingUser()`. **PIÈGE:** « le contexte utilisateur peut-il être usurpé via le chat ? » **ERREUR:** dire oui — non, il vient exclusivement du JWT vérifié.

Q35. Comment le Copilot sait-il qui est connecté ?

20S: le même token JWT que le reste de l'application, transmis à chaque appel API. **1MIN:** aucun second système d'identité pour l'IA. **TECHNIQUE:** Authorization: Bearer, intercepteur Dio partagé. **PIÈGE:** « le Copilot a-t-il son propre login ? » **ERREUR:** dire oui.

Q36. Comment les permissions utilisateur sont-elles propagées jusqu'aux tools ?

20S: `actingUser` est transmis de bout en bout, jusqu'au `PermissionManager` qui l'utilise pour chaque appel de Tool. **1MIN:** jamais recalculé, jamais un privilège élevé propre à l'IA. **TECHNIQUE:** `mcpToolAdapter.js`. **PIÈGE:** demander à tracer le chemin exact. **ERREUR:** perdre le fil JWT→`actingUser`→Tool.

Q37. Comment auditer une action exécutée par une IA ?

20S: chaque appel de Tool est audité (`McpServer._audit`) et chaque tour de conversation également (`AIManager._audit`). **1MIN:** deux journaux distincts, non fusionnés — limite connue. **TECHNIQUE:** `security/auditLog.js`. **PIÈGE:** « ces logs sont-ils centralisés ? » **ERREUR:** dire oui.

Q38. Une IA peut-elle supprimer une donnée CRM ?

20S: seulement si un Tool de suppression existe, autorisé pour le rôle, et jamais sans confirmation pour les agents à `PendingAction`. **1MIN:** ex. `delete_por_promesh` réservé admin-tier. **TECHNIQUE:** RBAC par Tool. **PIÈGE:** « une suppression peut-elle se faire sans confirmation ? » **ERREUR:** dire que TOUT Tool d'écriture passe par confirmation — faux, seuls 3 agents ont ce mécanisme (Commercial, Meeting, Automation).

Q39. Comment validez-vous les actions effectuées par un agent ?

20S: validation Joi de l'input + permission du Tool +, pour certains agents, confirmation explicite de l'utilisateur. **1MIN:** `AutomationAgent` est le seul où le LLM choisit lui-même le tool/input (via Planner), compensé par une restriction de rôle plus stricte. **TECHNIQUE:** `ValidationManager`. **PIÈGE:** « tous les agents sont-ils aussi stricts ? » **ERREUR:** dire oui — non, il y a une hétérogénéité assumée.

Q40. Quelle est la limite actuelle de votre architecture IA ?

20S: l'état conversationnel et le RAG sont volatiles (en mémoire), non partagés entre plusieurs instances serveur. **1MIN:** choix assumé de phase de développement, pas un oubli — l'architecture en interfaces permet un futur remplacement (pgvector, Redis) sans réécriture. **TECHNIQUE:** `InMemoryVectorStore`, `PendingActionStore`. **PIÈGE:** « c'est un vrai problème de production alors ? » **ERREUR:** minimiser — reconnaître que c'est une vraie limite pour un déploiement multi-instance, documentée honnêtement.

19. Questions pièges du jury

Q. Pourquoi ne pas utiliser une solution CRM existante (Salesforce, HubSpot) ?

Réponse honnête : le projet couvre un sous-système industriel propre (POR PROMESH, suivi de production, récupérables) qu'un CRM commercial générique ne modélise pas nativement ; adapter un CRM du marché à ces besoins aurait nécessité autant, voire plus, de développement custom que de partir d'une base propre.

Q. Pourquoi avez-vous développé votre propre CRM plutôt que d'utiliser un framework CRM open-source ?

Réponse honnête : non identifié dans les sources analysées — cette décision de départ relève du cahier des charges, pas du code ; je peux justifier l'architecture actuelle, pas la décision commerciale initiale.

Q. Pourquoi cette technologie et pas une autre ?

Réponse honnête : pour chaque choix (Express, Sequelize, Flutter, Ollama), je peux justifier la cohérence avec l'existant et les contraintes observées (multiplateforme, coût LLM) — je ne prétends pas que c'était objectivement le seul choix possible.

Q. Quelle est votre plus grosse faiblesse ?

Réponse honnête : le MFA, bien qu'entièrement implémenté et testé, n'est jamais déclenché par la connexion réelle — c'est une lacune de sécurité concrète, pas cosmétique.

Q. Quelle partie du projet avez-vous réellement développée ?

Réponse à adapter à la réalité personnelle du candidat — ce document ne peut pas répondre à la place du candidat sur sa contribution individuelle ; c'est une question hors périmètre du code analysé.

Q. Qu'est-ce qui est encore incomplet ?

Réponse honnête : MFA débranché, tests Flutter quasi absents (1 fichier), gestion d'erreur non centralisée, RAG/vector store volontairement minimalistes, pas de Docker.

Q. Quel est le risque principal ?

Réponse honnête : l'absence de 2FA effective malgré une infrastructure prête, combinée à la route /uploads non authentifiée — deux failles de sécurité concrètes et documentées.

Q. Que changeriez-vous aujourd'hui ?

Réponse honnête : brancher le MFA (l'effort technique est faible, les fonctions existent déjà) et unifier les trois strates de routage backend.

Q. Que feriez-vous avec plus de temps ?

Réponse honnête : un vector store persistant (pgvector) pour le RAG, une suite de tests Flutter significative, et la généralisation de `requireToolUse` à tous les agents à tools d'écriture.

Q. Pourquoi votre architecture est-elle suffisamment scalable ?

Réponse honnête : elle ne l'est pas totalement — l'état IA en mémoire (PendingAction, RAG) est un point de blocage documenté pour un déploiement multi-instance. Le reste du backend (Express + pool Sequelize) scale de façon plus classique.

Q. Pourquoi votre IA est-elle fiable ?

Réponse nuancée : elle n'est pas fiable à 100% (le LLM local peut mal suivre le protocole) — la fiabilité vient des garde-fous mécaniques (`requireToolUse`, RBAC, validation), pas du LLM lui-même.

Q. Que se passe-t-il si l'IA donne une mauvaise réponse ?

Réponse honnête : pour une question documentaire, une réponse imprécise peut passer (limite du RAG lexical) ; pour une action d'écriture, le pire cas est bloqué par la confirmation explicite et `requireToolUse` — jamais une écriture silencieuse incorrecte pour les agents concernés.

Q. Pourquoi utiliser l'IA ici alors qu'une requête SQL suffit ?

Réponse honnête : pour les questions simples, effectivement une requête suffirait — la valeur de l'IA est dans la formulation libre et l'enchaînement multi-étapes (ex. relance avec résolution de contact + commercial), pas dans chaque cas individuel.

Q. Pourquoi utiliser RAG plutôt qu'un simple moteur de recherche plein texte ?

Réponse honnête : le RAG ajoute une reformulation conversationnelle par le LLM ; un moteur plein texte (ex. PostgreSQL full-text

search) aurait pu suffire pour la recherche brute, mais n'aurait pas produit une réponse rédigée avec citations.

Q. Pourquoi ne pas faire du fine-tuning ?

Réponse honnête : coût et corpus insuffisant — non tenté, non documenté.

Q. Quelle est votre dette technique ?

Réponse honnête : documentée exhaustivement (12 items priorisés, section 18 du document technique) — MFA débranché, routes mortes, redondance email, absence de Docker, tests Flutter quasi nuls.

Q. Quelle est la partie la plus difficile du projet ?

Réponse honnête : garantir que le Copilot IA ne contourne jamais le RBAC existant tout en restant utile — équilibre entre sécurité stricte et flexibilité conversationnelle.

20. Choix technologiques

Technologie	Pourquoi choisie	Alternative	Pourquoi pas l'alternative
Flutter/Dart	Un seul code source pour 6 plateformes	React Native, apps natives séparées	Coût de maintenance multiplié par plateforme (non documenté comme comparé, mais cohérent avec le choix observé)
Node.js/Express	Écosystème JS homogène avec le kernel IA (LangChain)	Django, Spring, .NET	Aurait introduit un second langage backend
PostgreSQL	Relationnel robuste, JSONB/ENUM natifs	MySQL, MongoDB	Besoin de relations strictes (FK) peu adapté à un document store pur
REST API	Simplicité, large compatibilité client	GraphQL	Non nécessaire vu le nombre d'endpoints fixes par écran
JWT	Sans état, adapté au multi-plateforme	Sessions serveur	Nécessiterait un store de session partagé
PM2 (référéncé)	Gestionnaire de process Node.js standard	systemd, forever	Non documenté dans le dépôt
Ollama (LLM local)	Aucun coût d'API, exécution locale	OpenAI, Anthropic (implémentés, inactifs)	Coût récurrent par appel, dépendance réseau externe
RAG maison (hashing + mémoire)	Fonctionnel hors ligne, sans dépendance externe	Embedding neuronal + pgvector	Complexité d'infrastructure jugée non nécessaire à ce stade (choix de phase documenté)
Git / GitHub Actions	Standard de l'industrie, intégration CI simple	GitLab CI, Jenkins	Non comparé dans le dépôt

Technologies explicitement absentes (ne jamais les présenter comme existantes) : Docker, Nginx, OCR, base de données vectorielle dédiée.

21. « Pourquoi ? » — questions de soutenance

Pourquoi PostgreSQL ? Parce que le projet a besoin de relations strictes entre projets, contacts, demandes et fiches de production — un modèle relationnel garantit cette intégrité par des clés étrangères.

Pourquoi Flutter ? Parce qu'un seul code source devait couvrir web, mobile et desktop pour des utilisateurs terrain (atelier) et bureau.

Pourquoi REST ? Parce que chaque écran correspond à un nombre fixe et raisonnable d'appels — la flexibilité de GraphQL n'était pas nécessaire.

Pourquoi JWT ? Parce qu'une API sans état s'adapte mieux à plusieurs plateformes clientes qu'une session serveur.

Pourquoi RBAC ? Parce que 7 rôles réels et plusieurs comptes désignés par email doivent avoir des périmètres d'action strictement différents.

Pourquoi cette architecture (3 strates) ? Parce que le projet a évolué progressivement — les modules récents suivent un pattern en couches plus rigoureux, sans réécrire l'existant fonctionnel.

Pourquoi des services ? Pour séparer la logique métier de la couche HTTP, au moins dans les modules récents.

Pourquoi un repository ? Pour isoler l'accès aux données Sequelize du reste de la logique métier.

Pourquoi RAG ? Pour ancrer les réponses documentaires sur de vrais textes, jamais des données métier, avec des citations vérifiables.

Pourquoi un LLM ? Pour comprendre une formulation libre et rédiger une réponse naturelle, la détection d'intention elle-même restant déterministe.

Pourquoi un agent ? Pour isoler le prompt, les outils et les permissions par domaine métier, plutôt qu'un seul assistant générique.

Pourquoi des tools ? Pour que le LLM agisse sur de vraies données, validées et autorisées, jamais inventées.

Pourquoi des embeddings ? Pour mesurer la similarité entre une question et un document sans comparaison mot à mot exacte.

Pourquoi une vector database (en théorie) ? Pour chercher rapidement des passages similaires — ici remplacée par un stockage en mémoire volontairement simple.

Pourquoi ne pas utiliser uniquement SQL ? Parce qu'une requête SQL exige des mots-clés exacts, pas une formulation libre en langage naturel.

Pourquoi ne pas utiliser uniquement un LLM ? Parce qu'un LLM seul n'a aucune garantie de véracité sur une donnée métier précise.

Pourquoi ne pas utiliser du fine-tuning ? Parce que le corpus documentaire est trop restreint et le coût d'entraînement non justifié à ce stade.

Pourquoi cette organisation des modules ? Parce que chaque domaine métier (RH, maintenance, production) a ses propres règles et permissions, mieux isolées dans des dossiers dédiés.

Pourquoi cette séparation métier/technique ? Pour que la logique métier (règles, permissions) reste lisible indépendamment de la mécanique HTTP.

22. « Comment ? » — questions de soutenance

Comment fonctionne l'authentification ? Login vérifié par bcrypt, émission d'un JWT access (15 min) et d'un refresh (7 jours, cookie httpOnly).

Comment authentifiez-vous les requêtes API ? Un en-tête `Authorization: Bearer` vérifié par le middleware `authRequired` avant toute route protégée.

Comment sécurisez-vous les mots de passe ? Hachage bcrypt, jamais stockés ni loggés en clair.

Comment gérez-vous les permissions ? Une fonction unique `isAuthorized()`, appelée par tous les middlewares RBAC et par le module IA.

Comment stockez-vous les données de production ? POR PROMESH dans un schéma normalisé à 7 tables ; PROBAR/Mélange/Maintenance dans une table générique discriminée par ENUM.

Comment validez-vous une entrée utilisateur ? Joi pour les modules récents, validation manuelle pour le code legacy.

Comment l'IA détecte-t-elle l'intention d'un message ? Par comptage de mots-clés (déterministe), sans appel LLM à cette étape, complété par 7 mécanismes d'override contextuel.

Comment le RAG construit-il sa réponse ? Recherche cosinus sur des documents hachés en vecteurs, reranking lexical, puis reformulation par le LLM avec citations.

Comment les tools sont-ils appelés ? Le LLM répond en JSON structuré, interprété par ToolCallingChain, qui invoque le Tool via le protocole MCP.

Comment les permissions sont-elles propagées jusqu'aux tools ? Le même objet `actingUser`, dérivé du JWT, est transmis de l'orchestrateur jusqu'au `PermissionManager`.

Comment les données sont-elles exportées ? PDF via pdfkit, Excel via exceljs/xlsx, selon le module.

Comment les imports sont-ils traités ? Upload multer, validation de format, puis insertion en base (ex. import CSV clients).

23. « Que se passe-t-il si ? »

Scénario	Cause	Détection	Protection	Comportement
Le token expire	Dépassement de 15 min	401 sur la requête suivante	authRequired	Client nettoie le stockage local, pas de refresh automatique transparent
L'utilisateur n'a pas la permission	Rôle non autorisé	Middleware RBAC	requireRole/moduleAccessGuard	403 Forbidden
L'API retourne 403	Cf. ci-dessus	—	—	Message d'erreur explicite côté client
La base de données tombe	Panne PostgreSQL	Erreurs Sequelize	Pool de connexions (retry limité)	Toutes les requêtes échouent, 500 générique
Le LLM tombe	Ollama indisponible	Erreur réseau	classifyProviderHttpError, retry 2×	Message d'erreur générique sûr, jamais de crash
Le RAG ne trouve rien	Aucun chunk pertinent	citations vides	Instruction explicite du prompt	« Je ne trouve pas cette information dans la documentation »
Le LLM hallucine	Mauvais suivi du protocole JSON	steps.length==0	requireToolUse (3 agents)	Réponse rejetée, message honnête de repli
Le fichier uploadé est invalide	Extension/taille refusée	Filtre multer	Whitelist d'extensions	Erreur d'upload, requête rejetée
L'import échoue	Format CSV incorrect	Validation format	—	400 erreur, aucune donnée partiellement importée non confirmée dans le code
Une migration échoue	Erreur SQL/scéma	Sortie sequelize-cli	CI tolère l'échec (continue-on-error) — dette	Déploiement non bloqué automatiquement, risque de dérive
Deux utilisateurs modifient la même donnée	Écriture concurrente	Non identifié de verrou optimiste	—	Dernière écriture gagne (comportement Sequelize par défaut)
Un utilisateur tente d'accéder aux données d'un autre commercial	Absence de scoping	scopeInputForActingUser (IA), ownership inline (API)	Vérifié en base pour les écritures	Refus ou périmètre forcé
Un utilisateur appelle directement une API interdite	Contournement de l'UI	Middleware serveur (jamais côté client)	Toujours vérifié côté serveur	403, identique à un appel via l'UI
Un tool IA est appelé sans permission	Rôle insuffisant	PermissionManager avant exécution	isAuthorized()	PERMISSION_DENIED, jamais exécuté

24. Questions à partir des diagrammes

Diagramme de composants (architecture globale)

Q1. Pourquoi le kernel IA est-il un composant séparé du reste du backend ?

Ports & Adapters — dépendance unidirectionnelle vérifiée par des tests qui interdisent au CRM historique d'importer le kernel IA, jamais l'inverse.

Q2. Pourquoi trois styles de routes coexistent-ils sur le même diagramme ?

Évolution progressive du projet — les modules récents suivent le pattern en couches, le reste garde une logique legacy fonctionnelle mais non refactorisée.

Q3. Où se trouve la sécurité sur ce diagramme ?

Entre le client et les routes : `moduleAccessGuard` (global) puis `authRequired`, avant tout accès aux trois styles de routage.

Q4. Où se trouve la validation ?

Dans les validators `Joi` (modules récents) ou dans des fonctions inline (routes legacy) — jamais côté frontend seul.

Q5. Pourquoi cette dépendance entre `kanban.service.js` et `project.service.js` ?

Un couplage réel confirmé par l'audit préexistant, pour l'invalidation de cache — pas un cycle bloquant, mais une étoile de dépendances à surveiller.

Diagramme de séquence (ex. cycle de vie d'un projet)

Q1. Pourquoi cet ordre précis (validation avant persistance) ?

Pour ne jamais écrire une donnée invalide en base — la validation intervient toujours avant l'accès `Sequelize`.

Q2. Qui appelle qui exactement ?

Le frontend appelle la route, la route (ou son controller/service) appelle `Sequelize`, jamais l'inverse.

Q3. Où se produit l'erreur si le rôle est insuffisant ?

Au niveau du middleware `requireOwnerOrAdmin`, avant d'atteindre la logique de mise à jour.

Q4. Où se fait l'autorisation dans cette séquence ?

Juste après `authRequired`, avant le traitement métier — jamais après l'écriture.

Diagramme ERD

Q1. Pourquoi cette relation entre `Project` et `PipelineStage` ?

Un projet appartient à une seule étape à la fois (1-N depuis `PipelineStage`), reflétant le pipeline de vente réel.

Q2. Pourquoi cette cardinalité 1-N entre `PorPromesh` et ses sous-tables ?

Une fiche `PROMESH` peut avoir plusieurs contrôles qualité, arrêts machine, non-conformités — reflet direct du processus de production réel.

Q3. Pourquoi cette clé étrangère avec `onDelete CASCADE` sur `ArchiveRequestMessage` ?

Un message n'a de sens que rattaché à sa demande d'archivage — sa suppression est logique si la demande elle-même disparaît.

Q4. Pourquoi `IndustrialRecord` n'a-t-il qu'une seule table pour 3 modules ?

Choix de modélisation léger assumé, documenté dans le commentaire du modèle lui-même — pas un oubli.

Diagramme IA (architecture Copilot)

Q1. Pourquoi RAG à cet endroit précis du flux ?

Uniquement pour les questions documentaires (intent `documentation_question`) — jamais pour les données métier, qui passent par un Tool.

Q2. Pourquoi un Agent plutôt qu'un seul LLM générique ?

Pour isoler prompt, permissions et outils par domaine métier — un agent RH n'a jamais accès aux outils de production.

Q3. Pourquoi Tool Calling plutôt qu'une réponse texte directe ?

Pour que toute donnée métier affichée provienne d'une exécution réelle, vérifiable, jamais du texte généré seul.

Q4. Où est appliqué le RBAC sur ce diagramme ?

Deux fois : au niveau de l'agent (sélection) et au niveau du Tool (PermissionManager), jamais une seule fois.

Q5. Où est contrôlé l'accès aux données sur ce diagramme ?

Dans le service métier appelé par le Tool — jamais dans l'agent ni dans le LLM.

Diagramme PendingAction

Q1. Pourquoi une étape de confirmation explicite plutôt qu'une exécution directe ?

Pour ne jamais faire confiance à une décision LLM non supervisée sur une écriture réelle — né d'un principe de conception, renforcé par des incidents observés.

Q2. Pourquoi consume() est-il appelé avant l'exécution du Tool ?

Pour garantir l'atomicité — une seconde confirmation ne retrouve plus rien à exécuter, empêchant toute double écriture.

Diagramme RBAC

Q1. Pourquoi une seule fonction isAuthorized() plutôt qu'une par module ?

Pour garantir que la logique de permission n'est jamais dupliquée ni divergente entre le CRM et l'IA.

Q2. Pourquoi un modèle hybride rôle-ou-email ?

Pour représenter des comptes désignés (gestionnaire RH, maintenance) qui ne justifient pas un rôle ENUM dédié.

25. Simulation de soutenance

JURY : Présentez-nous votre projet en une phrase.

CANDIDAT : Un CRM pour CBI Tunisia qui centralise projets, prospection commerciale, RH, maintenance et production industrielle, avec un Copilot IA conversationnel intégré au même RBAC que le reste de l'application.

JURY : Qu'est-ce qui est le plus spécifique dans votre projet ?

CANDIDAT : Le sous-système industriel — en particulier le cycle de verrouillage POR PROMESH, où une fiche validée devient définitivement non modifiable.

JURY : Comment ce verrouillage est-il implémenté techniquement ?

CANDIDAT : Un champ booléen `isLocked` sur le modèle `PorPromesh`, vérifié côté service avant toute écriture — pas seulement une contrainte de base de données.

JURY : Votre CRM gère-t-il la facturation ?

CANDIDAT : Non. J'ai vérifié explicitement : aucun modèle Invoice ou Payment n'existe. Seul un import de fichier « factures » basique est présent côté clients.

JURY : Pourquoi ne pas l'avoir développé ?

CANDIDAT : Cela relève du périmètre initial du projet, non documenté dans le code — je ne peux pas justifier une décision de scope que le code ne porte pas.

JURY : Comment le Copilot IA empêche-t-il une fuite de données entre commerciaux ?

CANDIDAT : Une fonction `scopeInputForActingUser()` force le périmètre de lecture à l'utilisateur courant pour tout rôle non admin-tier, quel que soit ce que le LLM a pu fournir en paramètre.

JURY : Cela a-t-il toujours été le cas ?

CANDIDAT : Non — un incident réel a été documenté dans le code lui-même : avant ce correctif, une hallucination du LLM avait exposé des projets d'autres utilisateurs.

JURY : Pourquoi votre MFA n'est-il pas actif ?

CANDIDAT : L'infrastructure complète existe et est testée, mais les fonctions qui déclencheraient le challenge à la connexion ne sont appelées par aucun autre code — un commentaire du fichier `auth.routes.js` le confirme explicitement.

JURY : N'est-ce pas une faille grave ?

CANDIDAT : Oui, je le reconnais comme une lacune de sécurité réelle et documentée, avec une correction identifiée et peu coûteuse à appliquer.

JURY : Pourquoi Ollama plutôt qu'OpenAI ?

CANDIDAT : Ollama tourne localement, sans coût d'API ni dépendance réseau externe. OpenAI et Anthropic sont pleinement implémentés et testés dans le code, mais non activés dans la configuration actuelle.

JURY : Que faudrait-il pour basculer ?

CANDIDAT : Changer une seule variable d'environnement (`AI_LLM_PROVIDER`) et fournir une clé API — aucune modification de code n'est nécessaire, grâce à l'abstraction `ILlmProvider`.

JURY : Le Copilot peut-il halluciner ?

CANDIDAT : Oui, et je peux le démontrer : un incident réel a été observé sur `PorPromeshAgent`, où le modèle a répondu sans avoir appelé l'outil nécessaire.

JURY : Comment l'avez-vous corrigé ?

CANDIDAT : En introduisant `requireToolUse`, qui rejette toute réponse finale si aucun outil n'a réellement été exécuté — fondé uniquement sur l'état réel de la boucle, jamais sur le texte du modèle.

JURY : Qu'est-ce qu'un embedding dans votre projet ?

CANDIDAT : Un vecteur de 256 dimensions produit par un algorithme de hachage déterministe (feature hashing), pas par un modèle neuronal.

JURY : Pourquoi ne pas avoir utilisé un vrai modèle d'embedding ?

CANDIDAT : Choix de phase assumé, documenté dans le code lui-même — l'architecture en interfaces permet de brancher un vrai modèle plus tard sans réécrire le reste du pipeline RAG.

JURY : Comment le rôle « responsable_logistique_achat » est-il restreint ?

CANDIDAT : Un middleware global, `moduleAccessGuard`, limite ce rôle à une liste fixe de préfixes d'URL, avant même l'authentification fine de chaque route.

JURY : Pourquoi seulement ce rôle ?

CANDIDAT : C'est un choix produit confirmé dans le code — les autres rôles ne sont restreints qu'au niveau des routes/services, pas à ce niveau grossier.

JURY : Que se passe-t-il si deux utilisateurs valident la même fiche POR PROMESH simultanément ?

CANDIDAT : Non identifié de mécanisme explicite de verrou optimiste dans le code analysé — c'est une limite honnête que je préfère reconnaître plutôt que d'inventer une protection non vérifiée.

JURY : Pourquoi 76 outils IA et pas un chiffre rond documenté ailleurs ?

CANDIDAT : Les commentaires internes du code citent des chiffres plus anciens (69, 73, 74) — j'ai vérifié le nombre réel en exécutant le code moi-même plutôt qu'en faisant confiance aux commentaires, qui sont devenus obsolètes au fil des ajouts.

JURY : Le Copilot a-t-il accès à toute la base de données ?

CANDIDAT : Non, jamais directement. Chaque accès passe par un Tool précis, avec un schéma d'entrée validé et une permission vérifiée — il n'existe aucun chemin générique « accès complet ».

JURY : Comment gérez-vous la confidentialité entre les demandes RH ?

CANDIDAT : Seul le demandeur et le gestionnaire RH désigné (rôle admin-tier + email spécifique) peuvent voir/traiter une demande — vérifié par le middleware `requireHrReviewer`.

JURY : Qu'arrive-t-il si le serveur Ollama est éteint pendant une démonstration ?

CANDIDAT : L'erreur est classifiée (indisponibilité), un message générique et sûr est renvoyé à l'utilisateur — jamais un crash, jamais une fuite technique.

JURY : Pourquoi ne pas avoir mis toutes les routes dans le même style architectural ?

CANDIDAT : Le legacy fonctionne et est déjà testé en partie — le réécrire entièrement représentait un risque de régression non justifié par le calendrier du projet.

JURY : Qu'est-ce qui distingue Maintenance Requests d'Industrial Records (maintenance) ?

CANDIDAT : Le premier est une demande interne de panne (équipement de bureau/site), le second une fiche de maintenance d'atelier industriel — deux tables, deux permissions distinctes, jamais confondues dans le code.

JURY : Le RAG peut-il un jour lire les données clients par erreur ?

CANDIDAT : Structurellement non — SourceGuard bloque toute extension non documentaire et tout chemin sous `src/models`, `src/modules`, etc., avec une vérification anti-symlink en plus.

JURY : Que recommanderiez-vous en priorité avant une mise en production ?

CANDIDAT : Brancher le MFA, authentifier la route `/uploads`, et ajouter un middleware d'erreur centralisé — les trois corrections les moins coûteuses avec le plus d'impact sécurité.

JURY : Pour conclure, quel est votre argument le plus fort ?

CANDIDAT : Que chaque affirmation de ce document est reliée à un fichier, une ligne ou un test réel — je n'ai rien présenté qui ne soit vérifiable directement dans le dépôt.

26. Présentation orale (30 secondes par sujet)

Présentation du projet : Un CRM pour CBI Tunisia couvrant projets, commercial, RH, maintenance et production industrielle, avec un Copilot IA conversationnel intégré au même système de permissions.

Problématique : Centraliser des processus métier dispersés, avec un contrôle d'accès cohérent par rôle, plutôt que des écrans isolés à logique de permission propre.

Besoins spécifiques : Le cycle de verrouillage POR PROMESH, la table générique de production PROBAR/Mélange/Maintenance, et le suivi déchet/produit fini des Récupérables.

Besoins génériques : Authentification, RBAC, notifications, export, upload — le socle indispensable à tout le reste.

Architecture : Backend Express/Sequelize en trois strates historiques, frontend Flutter multiplateforme, kernel IA séparé en Ports & Adapters.

Base de données : PostgreSQL, ~44 modèles Sequelize, migrations exclusivement versionnées, jamais de synchronisation automatique.

Sécurité : JWT + RBAC unifié partout, avec deux lacunes assumées : MFA débranché et route de fichiers non authentifiée.

Production : Deux modélisations coexistent — POR PROMESH normalisé à 7 tables, et une table générique pour PROBAR/Mélange/Maintenance.

Recoverables : Suivi du déchet et du produit fini par machine/ligne/poste/date, avec une contrainte d'unicité garantissant une seule fiche par combinaison.

Finance : Non implémenté — seul un import basique de fichier existe, sans module transactionnel.

IA : 19 agents spécialisés, chacun délégrant au même RBAC que le reste du CRM, jamais de privilège propre.

Copilot : Un point d'entrée conversationnel unique, avec confirmation explicite obligatoire pour toute action d'écriture proposée.

RAG : Réponses documentaires ancrées sur de vrais textes internes, jamais sur des données métier, avec citations vérifiables.

LLM : Ollama local (qwen3:8b), sans coût d'API, avec deux autres providers déjà implémentés mais inactifs.

Agents : Chacun isole un domaine métier avec son propre prompt, ses propres outils autorisés et son propre RBAC.

Tools : 76 outils validés (Joi) et autorisés (PermissionManager) avant toute exécution — jamais un accès direct à la base.

Résultats : Un système fonctionnel, testé (500+ scénarios IA), avec des protections anti-hallucination nées d'incidents réels observés et corrigés.

Limites : MFA débranché, RAG/vector store volontairement minimalistes, état IA non partagé entre instances, tests Flutter quasi absents.

Perspectives : Vector store persistant, MFA branché, généralisation de requireToolUse, conteneurisation Docker.

27. Réponses à ne pas donner

NE PAS DIRE : « J'ai utilisé cette technologie parce qu'elle est moderne. »

PRÉFÉRER : « Express/Sequelize répond à la contrainte d'un backend JavaScript homogène avec le kernel IA, tout en donnant accès à un ORM relationnel mature. »

NE PAS DIRE : « L'IA est totalement fiable, elle ne se trompe jamais. »

PRÉFÉRER : « L'IA peut se tromper — c'est pourquoi des garde-fous mécaniques (requireToolUse, RBAC) ne dépendent jamais de sa fiabilité propre. »

NE PAS DIRE : « Le MFA fonctionne, on peut le tester en direct. »

PRÉFÉRER : « Le MFA est implémenté et testé isolément, mais pas branché sur le flux de connexion réel — je peux le montrer dans le code, pas en démonstration live. »

NE PAS DIRE : « Le RAG utilise une vraie base vectorielle comme Pinecone. »

PRÉFÉRER : « Le RAG utilise un stockage en mémoire volatile avec un embedding par hachage — un choix de phase assumé, pas une base vectorielle dédiée. »

NE PAS DIRE : « Il n'y a aucune dette technique dans ce projet. »

PRÉFÉRER : « J'ai identifié et priorisé 12 éléments de dette technique, dont deux critiques pour la sécurité. »

NE PAS DIRE : « Le CRM gère la facturation, mais je n'ai pas eu le temps de le montrer. »

PRÉFÉRER : « Aucun module de facturation n'existe dans le code — c'est hors du périmètre implémenté, pas une fonctionnalité cachée. »

NE PAS DIRE : « Tous les agents utilisent requireToolUse. »

PRÉFÉRER : « Seuls 3 agents (Project, Commercial, PorPromesh) l'utilisent aujourd'hui, sur les 19 agents actifs. »

NE PAS DIRE : « Le système est prêt pour la production à grande échelle. »

PRÉFÉRER : « Le système est fonctionnel pour une démonstration ; une mise à l'échelle horizontale nécessiterait de résoudre la volatilité de l'état IA et l'absence de Docker. »

NE PAS DIRE : « J'ai choisi PostgreSQL parce que c'est la meilleure base de données. »

PRÉFÉRER : « PostgreSQL répond au besoin de relations strictes entre les entités du projet, avec un support natif JSONB/ENUM utile pour certains champs. »

NE PAS DIRE : « L'IA a accès à toute la base de données pour être plus utile. »

PRÉFÉRER : « L'IA n'a accès qu'aux données exposées explicitement par un Tool validé et autorisé — jamais un accès direct ou générique. »

NE PAS DIRE : « Il n'y a pas de faille de sécurité dans ce projet. »

PRÉFÉRER : « J'ai identifié des failles précises (MFA débranché, route /uploads non authentifiée) et je peux expliquer pourquoi et comment les corriger. »

NE PAS DIRE : « J'ai tout testé, la couverture est complète. »

PRÉFÉRER : « Le kernel IA est couvert à environ 92% ; le frontend Flutter, en revanche, n'a qu'un seul fichier de test — une faiblesse assumée. »

NE PAS DIRE : « PROMESH et PROBAR sont gérés de la même façon dans la base. »

PRÉFÉRER : « Non, POR PROMESH a un schéma normalisé à 7 tables, tandis que PROBAR partage une table générique avec Mélange et Maintenance. »

NE PAS DIRE : « Le Copilot peut tout faire à la place de l'utilisateur. »

PRÉFÉRER : « Le Copilot agit toujours au nom de l'utilisateur authentifié, avec ses permissions exactes, jamais un privilège supplémentaire. »

NE PAS DIRE : « L'architecture DDD a été appliquée à ce projet. »

PRÉFÉRER : « Le projet ne suit pas formellement le DDD ; j'ai analysé les contextes bornés observés de fait, sans agrégats ni value objects explicites. »

NE PAS DIRE : « Toutes les routes sont sécurisées de la même façon. »

PRÉFÉRER : « La majorité des routes passent par authRequired, mais certaines (comme /utils/expand-maps) ne sont pas protégées — une dette identifiée. »

NE PAS DIRE : « J'ai comparé objectivement toutes les alternatives technologiques possibles. »

PRÉFÉRER : « Je peux justifier la cohérence des choix effectués avec les contraintes observées, sans prétendre à une étude comparative exhaustive documentée. »

NE PAS DIRE : « Le nombre d'outils IA est de 73 (ou 74). »

PRÉFÉRER : « 76, vérifié en exécutant le code — les commentaires internes citent des chiffres plus anciens, devenus obsolètes. »

NE PAS DIRE : « Le refresh token est totalement sécurisé contre le vol. »

PRÉFÉRER : « Il est haché en base et transmis en cookie httpOnly, mais sans rotation ni détection de réutilisation — une limite assumée. »

NE PAS DIRE : « Je ne sais pas » sans complément, face à une question sur une info absente du code.

PRÉFÉRER : « Non identifié dans les sources analysées » — formule qui montre une méthode rigoureuse plutôt qu'une lacune de préparation.

NE PAS DIRE : « L'IA fine-tune un modèle personnalisé pour ce projet. »

PRÉFÉRER : « Aucun fine-tuning n'existe — le modèle Ollama est utilisé tel quel, avec RAG et prompts personnalisés par agent. »

28. Fiche de révision finale

TOP 20 besoins spécifiques

1. Verrouillage POR PROMESH (isLocked)
2. Table polymorphe IndustrialRecord (probar/melange/maintenance)
3. Suivi déchet/produit fini par poste (RecuperableLigne)
4. Unicité fiche Récupérables (contrainte composite)
5. Rôle responsable_logistique_achat à périmètre restreint
6. Comptes désignés par email (RH, maintenance, root admin)
7. Confirmation explicite avant écriture Copilot (PendingAction)
8. requireToolUse anti-hallucination
9. Synchronisation Task+Google Calendar pour relances/réunions IA
10. Auto-crédation d'action selon l'étape du pipeline
11. Cycle d'approbation Archive Requests (root admin seul)
12. Cycle d'approbation HR Requests (gestionnaire désigné)
13. Cycle d'approbation Maintenance Requests (gestionnaire désigné)
14. Scoping RBAC des KPI commerciaux personnels
15. Extraction déterministe des créneaux de relance (jamais le LLM)
16. SourceGuard (RAG ne lit jamais les données métier)
17. Double porte d'accès au Copilot (feature-gate + RBAC fin)
18. Idempotence de synchronisation calendrier (upsertSync)
19. ADMIN_TIER_ROLES comme convention transverse répétée
20. Journal d'activité append-only par domaine (Project, HR)

TOP 20 besoins génériques

1. Authentification JWT
2. RBAC multi-niveaux
3. Gestion des utilisateurs
4. MFA (partiel)
5. Notifications (temps réel + persistées)
6. Pagination
7. Export PDF
8. Export Excel
9. Upload de fichiers
10. Responsive multi-device
11. Recherche/filtres
12. Dashboard/KPI
13. Historique d'activité
14. Reset de mot de passe
15. File d'attente email
16. Logs applicatifs
17. Import CSV
18. Rate limiting (partiel)
19. Validation des entrées (Joi)
20. Gestion des erreurs HTTP

TOP 20 règles métier

Voir section 9 (RB-001 à RB-010) — complété par : verrouillage irréversible, ownership systématique sur écriture sensible, activation de compte obligatoire, changement de rôle réservé, déduplication des rappels, confirmation IA obligatoire, anti-hallucination fondée sur l'état réel, restriction de préfixe par rôle, contrainte d'unicité Récupérables, comptes désignés par email.

TOP 20 choix techniques

Flutter (multiplateforme), GetX (état dominant), GoRouter, Dio, Node.js/Express, Sequelize, PostgreSQL, JWT (access+refresh), bcrypt, Joi, multer, Nodemailer, node-cron, Socket.IO, Ollama (LLM local), LangChain, MCP (protocole maison), hashing trick (embedding), InMemoryVectorStore, GitHub Actions (CI backend).

TOP 20 questions jury (toutes catégories)

Voir sections 14-17 pour la liste complète (110 questions) — les plus probables : pourquoi cette architecture, comment fonctionne le RBAC, pourquoi PostgreSQL, comment le MFA fonctionne-t-il réellement, quelle est votre plus grande faiblesse.

TOP 20 questions IA

Voir section 18 (40 questions) — les incontournables : pourquoi RAG, pourquoi Ollama, comment empêchez-vous une fuite RBAC via l'IA, comment gérez-vous les hallucinations, qu'est-ce qu'un embedding ici, pourquoi pas une vraie vector database, combien d'agents/tools exactement.

TOP 10 questions pièges

Voir section 19 — les plus dures : « pourquoi ne pas avoir utilisé un CRM existant », « quelle est votre plus grosse faiblesse », « le MFA fonctionne-t-il vraiment », « votre IA est-elle fiable », « quelle est votre dette technique ».

TOP 10 réponses essentielles

1. 7 rôles réels, aucun rôle Finance.
2. MFA implémenté mais jamais déclenché — assumé honnêtement.
3. 19 agents, 76 outils IA (vérifié par exécution).
4. `isAuthorized()` unique, jamais dupliquée.
5. RAG = hashing + mémoire volatile, pas de vraie vector DB.
6. `requireToolUse` sur 3 agents seulement.
7. `PendingAction` sur 3 agents (Commercial, Meeting, Automation).
8. Aucun module Finance, aucun OCR.
9. Ollama actif, OpenAI/Anthropic implémentés mais inactifs.
10. Chaque réponse de ce document est reliée à un fichier réel.

29. Réponses personnalisées au projet

Chaque réponse importante de ce document (sections 2, 3, 6, 9, 13, 18) cite déjà explicitement, quand c'est pertinent : le **module** concerné, le **fichier** ou composant (ex. `toolCallingChain.js`, `isAuthorized()`), l'**API** concernée (ex. `POST /por-promesh/:id/validate`), la **table** concernée (ex. `recuperable_fiches`) et le **workflow** associé (section 13). Cette discipline permet, si le jury demande « montrez-moi où c'est dans le code », de retrouver immédiatement l'élément exact — jamais une affirmation qui ne pourrait pas être démontrée en direct dans le dépôt.

30. Conclusion

- A. Ce qui rend le projet spécifique :** le sous-système industriel (POR PROMESH verrouillé, table polymorphe PROBAR/Mélange/Maintenance, suivi déchet/produit fini) et les garde-fous propres à l'intégration IA (confirmation explicite, requireToolUse).
- B. Ce qui est générique :** authentification, RBAC, notifications, export, upload — le socle indispensable mais non différenciant.
- C. La valeur métier :** un point d'accès centralisé à des processus auparavant dispersés, avec traçabilité et permissions cohérentes.
- D. La valeur technique :** un kernel IA construit en Ports & Adapters, vérifié mécaniquement par des tests, largement au-dessus de la rigueur du reste du code.
- E. La valeur de l'IA :** une réduction de la charge de navigation manuelle, avec des protections concrètes (RBAC partagé, confirmation explicite, anti-hallucination) nées d'incidents réels et corrigés, pas d'une conception théorique.
- F. Les limites :** MFA débranché, route de fichiers non authentifiée, RAG/vector store volontairement minimalistes, état IA non partagé entre instances, tests Flutter quasi absents.
- G. Les améliorations futures :** brancher le MFA, authentifier les uploads, vector store persistant, généraliser requireToolUse, conteneurisation Docker.
- H. Les arguments les plus forts pour la soutenance :** la rigueur méthodologique (chaque affirmation reliée à un fichier ou un test réel), la richesse et la sécurité du sous-système IA, et la capacité à distinguer sans détour ce qui existe, ce qui est partiel, et ce qui est absent.

CRM_Analyse_Besoins_et_Preparation_Soutenance — Projet PROBAR — 29 août 2026

Support de révision fondé exclusivement sur l'analyse directe du code source et les audits techniques précédents de ce même dépôt.